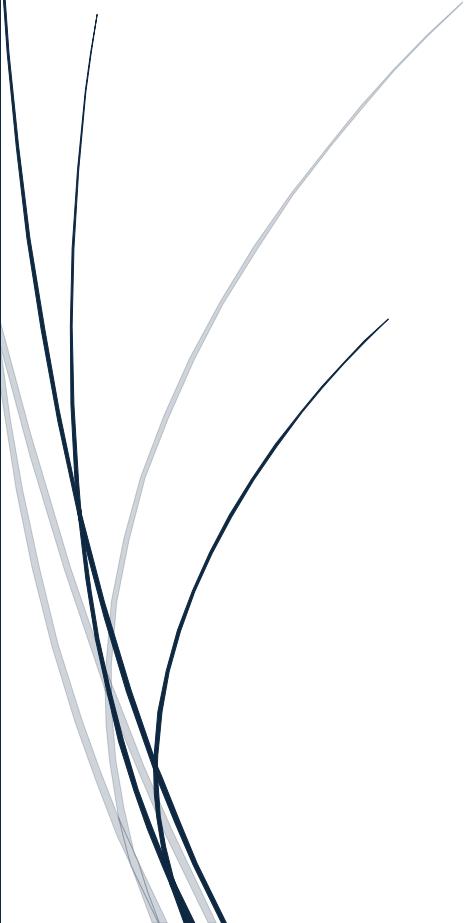




2/18/2026

CS1813 SOFTWARE IMPLEMENTATION

ASSIGNMENT 3 – NOUGHTS & CROSSES



Albert Guo - 2523488

BRUNEL UNIVERSITY LONDON 2025/26

CONTENTS

CONTENTS	1
IMPLEMENTATION SUMMARY	2
DESIGN SPECIFICATION/IMPLEMENTATION CHANGES	5
EVALUATION (BLACK BOX TESTING)	8
FULLY UPDATED SOURCE CODE LISTINGS	14
REFERENCES	22
OpenAI ChatGPT 5.1 mini – <i>noughts and crosses</i> . This answer is generated by AI. Available on https://chatgpt.com/c/693708ca-c110-832c-a638-a89209a49c35 (Generated 9 February 2025). See appendix A for further information.	
Appendix A (ChatGPT)	0
Appendix B (Claude AI)	30

IMPLEMENTATION SUMMARY

Below is the improved software requirements specification. In the improved version, every requirement (functional, non-functional and additional functionalities), has been thoroughly checked against the task brief and improved upon the LLM-generated version, which contains many errors, missing requirements and is disorganized with no visible numbering system.

2) FUNCTIONAL REQUIREMENTS

2.1) PHYSICAL & ASCII GAMEBOARD DETAILS & GAME PIECES:

- a) a physical board for the swiftbot to move on
- b) The physical board will be a 3x3 (9 squares), an area of 75 by 75cm and will be numbered [1-3]. Each square will measure 25x25 cm.
- c) each player will have 4 game pieces each: X or O, to mark their chosen position.
- d) There will be a command-line ASCII representation of the 3x3 board digitally for the user to execute their moves, with clear row/column labels mapping out the coordinates of moves.
- e) All inputs and any outputs should be handled by the command-line interface.

2.2) ALIGNING BUTTONS WITH CORRESPONDING LED COLOURS:

- a) Button A (front left): Red
- b) Button B (back left): Green
- c) Button X (front right): Blue
- d) Button Y (back right): Yellow

2.3) DISPLAY WELCOME SCREEN & USER REGISTRATION:

- a) The program will need to display a welcome screen with a user-readable message. It should have a brief description of the game rules, which must also be clear.
- b) The program shall begin the registration sequence when the user presses the SwiftBot 'A' button.
- c) the program should prompt the user to enter a username via the keyboard. The name should be in a string format; any other formats is invalid.
- d) a name will be randomly generated and assigned to the swiftbot after the user enters a username which can be configured manually.

2.4) GENERATE & ROLL DICE:

- a) The program will roll a virtual 6-sided die for both the player and swiftbot automatically.
- b) The player with the highest dice roll goes first and will be assigned the 'O' piece, whilst the other will have the 'X' piece.
- c) if there's a tie, the system shall re-roll until a single higher value is produced and the starter is determined.

2.5) USER'S TURN:

- a) For the user's turn, the System shall prompt the User to enter a move in the format [row, column] within the range 1-3 in full integers.
- b) The System shall validate and verify that the user input is numeric and in the right format.
- c) If invalid, the System shall show an error and re-prompt.
- d) The System shall check whether the selected square is empty

- e) if a square is an unavailable, the System shall notify the User and re-prompt for a different move.
 - f) When the User provides a valid move, the System shall mark the internal board state with the User's piece ('O' or 'X') on the square.
 - g) The System shall ensure that the user cannot make consecutive moves
 - h) update the displayed ASCII board after the move is successful
 - i) displays the following message: **'[User – O/X] moved to square [r, c]'** after each successful move
- 2.6) SWIFTBOT'S TURN:
- a) When it's the swiftbot's turn, the robot will randomly select its move to its target square within the range
 - b) display the chosen square to the User *before* instructing the SwiftBot to move.
 - c) After going to the chosen square, the swiftbot will then execute a blink sequence in green 3 times upon arriving to the target square.
 - d) The swiftbot will then return to its starting position in the board after the move.
 - e) the system should then display the move summary for swiftbot move:
[SwiftbotName – Piece] moved to square [r, c]
 - f) the swiftbot should keep track of its current location, orientation and decide the direction it would move to before going to that square
 - g) swiftbot should move in a suitable speed along the 75x75 physical board, so that it stays within squares for each move.
- //CHANGED FUNCTIONAL REQUIREMENTS: added swiftbot calibration details*
- 2.7) WIN/DRAW CONDITIONS, TRACING WINNING LINE & BLINKS LEDs:
- a) If the user wins, the swiftbot will trace the winning line and blinks green 3 times before and after tracing for an 'O' win.
 - b) If the swiftbot wins, the swiftbot will trace the winning line and blinks red 3 times before and after tracing for an 'X' win.
 - c) If it's a draw, the swiftbot will spin once and blink in blue 3 times before and after the move.
- 2.8) MAINTAINING LOG FILE & UPDATE BOARD:
- a) After each round (win or draw), the System shall append the following information in the log file: round number, the playing piece (i.e., 'O' or 'X'), the move positions (row and column), game result (win/draw), the winner player name or draw and current date & time.
 - b) The System shall update and maintain the scoreboard across rounds to track user performance
 - c) display the scoreboard results after each round.
 - d) When the user quits, the System shall close the log file and terminate the program
 - e) When the user or swiftbot wins a round, they will be awarded a score
- 2.9) PLAY AGAIN/QUIT OPTIONS:
- a) **Button Y:** play another round and the system will reset the board and start a new game
 - b) **Button X:** quit and terminate the program

3) NON-FUNCTIONAL REQUIREMENTS

- 3.1) **USABILITY:**
 - a) The game interface will be implemented as a simple command line interface console that is easy to navigate and intuitive.
 - b) Console UI formatting and design structure should be clear, simple and organised, displaying clear information.
 - c) Error messages should be clear, concise and on to the point. Non-technical users should not see that confusing paragraph/block of error message.
 - d) Displays game board in console game in clear and precise format.
- 3.1) **PERFORMANCE**
 - a) The system response time should be shorter from 1-2 seconds for updating player moves, calculating positions, updating boards.
- 3.2) **RELIABILITY**
 - a) There will be effective and reliable input validation to check the format, data type and numeric ranges for user input.
 - b) Efficient error-handling to manage exceptions, prevent unexpected outcomes like crashes and bugs and handle it gracefully.
- 3.3) **PORTABILITY**
 - a) The program should run on multiple platforms that is java-compatible and supports java runtime environment
- 3.4) **MAINTAINABILITY**
 - a) Code comments should be provided to explain each method's functionalities and purpose.
 - b) Code should be modular, which each block of code representing a specialised method performing a single function
- 3.5) **SCALABILITY**
 - a) The system should only have two players – the user and the swiftbot, as this is a two-player game.
 - b) A log file that consistently tracks and records game events and actions like player names, scoring, player positions, movements, wins, losses and draws.
- 3.6) **SECURITY**
 - a) the game doesn't handle sensitive data and therefore does not need any security strategies and backups to handle such.
- 3.7) **USER INTERFACE**
 - a) All inputs and outputs will be handled via the command-line console window
 - b) The user will interact with the system via the swiftbot buttons and the physical keyboard.
 - c) The system will handle swiftbot movements during each round
 - d) Swiftbot LEDs will light up depending on the button pressed, for example, pressing button 'A' will activate the red LED.
 - e) The user is expected to input their moves in an integer format and should be within the appropriate range of the board.

1) ADDITIONAL FUNCTIONALITIES

- 4.1) Choicdo the last move the player makes but has a limit of 2 times (including the players first move) to prevent them from too easily gaining an advantage and allow them to think quicker and more strategically to balance out the odds of the game.

- 4.2) a timer will be set where the user will have to put their piece on the game board and will allow the player to adjust from 5-10 seconds the time the program will recognise the validity of the move. After that time, the move will become invalid and will give that turn to the next player.
- 4.3) The game will have a difficulty settings option at the start of the game, in the introduction, where the command line interface will display options, which are 'easy', 'medium' and 'hard' after the introduction of the game, asking the user '*what game mode? Enter easy, medium or hard*'. the difficulty of the game will depend on the two functionalities stated above:
 - a) How much '**undo's**' the user will have depending on game difficulty. For instance, the 'easy' mode will have 2 new move undo's, meaning the user can revise their move choices back by removing their piece twice. Whereas the medium and hard modes will have 1 and none respectively.
 - b) This will prompt the program to ask, '**do you want to undo your last move, enter yes or no?**', if yes, the gameboard will remove your last piece to start again or if no, then the pieces stays there.
 - c) The program will keep track of the number of times the user uses the undo option based on their difficulty. This value will be enclosed in brackets like '**Undo move chances: 2**' after the program asks yes or no. this will be shown in the command line as '**do you want to undo your last move, enter yes or no? (Undo move chances: 2)**',
 - d) For each turn, a timer countdown will be enforced as a time limit the user is allowed during their move. For the easy mode, there won't be a timer but a time limit of 10 seconds for the medium mode and 5 seconds for the hard mode will be applied.

IMPLEMENTATION DETAILS

The implementation will include and will be based on all the functional and additional requirements, including following the principles of the non-functional requirements thoroughly. Based on the improved SRS, most of the core functionalities of the swiftbot has been implemented, although specific calibration details must be adjusted for the swiftbot to move along the physical board. However, certain aspects of the non-functional requirements like input validation aren't consistently applies throughout the program, and additional functionalities are applied accordingly but they may not visually come out as intended, such as the timer functionality for the medium and hard modes of the game but they print out the countdown on a new line every time, which can cause frustration with entering the user moves.

DESIGN SPECIFICATION/IMPLEMENTATION CHANGES

So far, the implementation goes pretty much as planned that the design specifications states. However, due to time constraints and prior complexity and somewhat disorganized appearance of my improved UI screens designs, it decided to use effective colour coding using ASCII and labelling to distinguish different types of outputs, like error and warning messages. Other slight minor changes were made to the program architecture itself that is not exclusively mapped to the flowchart exactly. For instance, I integrated the comparison of the player dice roll results into the broader 'generate and roll dice' method and calls either user or swiftbot turn. Other

than that, the implementations follow exactly and these minor changes don't affect the overall performance, visibility or behaviour of the program.

For the finished implementation itself, I had updated a certain few things; for instance, I had split the original one class into 3 separate classes, all in the same file. This is needed for abiding by the principle of single responsibility and to organize related methods within each "parent" class. The *CoreGameLogic* main class handles the heart of the game logic with all its private variables and objects being temporarily passed as copies to parameters of other class methods, ensuring the originals are never accessed and enforces encapsulation. (see *code listings below*)

For the implementation of the timer for the medium and hard modes, I had to change the timer by removing the countdown feature and replacing it with a timer thread that sleeps in the background for the fixed countdown duration and then comes back and tells the user their turn is either forfeited or not. Since the timer glitches sometimes and the cursor keeps glitching back to position 0 that renders the user practically unable to enter their move, a threaded timer would be a more reliable in this case, but one downside is that it doesn't show the live countdown which will make it harder to know if the timer is up however I had adjusted the timers for medium and hard modes to 20 and 10 seconds respectively, knowing that I had to account for the time taken for program processes and giving the user enough time to made their move with considering thinking strategies.

IMPLEMENTED REQUIREMENTS CHECKLIST SUMMARY

Requirement number	Requirement (functional/non-functional/additional)	Implemented the feature?	Reasons otherwise (issues found or changing certain features)	Any changes made
1	The program starts with a welcome message and displays game rules	Yes	None	None
2	The program shall begin the registration sequence when the user presses the SwiftBot 'A' button.	Yes	None	None
3	The program should start the registration by asking the user for a username and then auto-generates a username for the swiftbot.	Yes	none	None
4	The program should roll a 6-sided dice pseudo randomly for both players.	Yes	None	None
5	The system will prompt the user to enter a move in the format '[row, col]	Yes	None	None
6	It will apply input validation to the move to ensure it's in the correct format, rejecting invalid ones and prevention of consecutive moves	Yes	none	None
7	the program will assign the 'O' piece to the highest dice roller whilst other gets 'X'.	Yes	None	None
8	The program should update the board for every move and every round	Yes	None, except earlier issues where the board is blank during some program runs.	None
9	The swiftbot should move according to its selected move and target quare within the boards range and display them	Yes	None	None
10	The swiftbot will blink the appropriate LED colours whether the user or swiftbot wins and trace a winning line.	Yes	none	None

11	A log file will be maintained to record round results, display the results, increment both the round and the score	yes	None	None
12	The system should respond to two button events, X for quit or Y for restarting a new round without issues	Yes	None	None
13	The user interface should be clean, easy to use and organized	Partially, used ascii color coding for organizing different outputs, the overall interface could be better	None	None
14	Error messages should be clear, concise and include instructions (like terminate/re-prompt) if user made a mistake.	Yes, used consistent ascii colouring	none	None
15	Game performance should be optimal with less than 3 seconds lag for each output, such as updating boards.	Yes, the lag is almost zero seconds	none	None
16	user validation/verification should be applied across the program architecture consistently, like numeric checks.	Partially, some subtle aspects of the program where validation doesn't apply.	None	none
17	The program should run on multiple platforms that supports java runtime environment and some ascii coding	Yes, the program runs successfully on the university machines	none	None
18	For maintainability, code comments should be applied at the class, method and variable level to highlight the general purpose of each function and their part in the program	Yes, the comments are fairly and consistently applied across the program, however, there're very few many method variables and imported objects that are still not commented.	none	None
19	The program architecture should be modular, reflecting the single-responsibility principle and follow OOP principles.	Yes, it follows both of these principles and is divided into 3 static classes in the same file, but it doesn't use inheritance/polymorphism.	Now used 3 static classes instead of 1. I've grouped related methods under each common static class within the same java file, reflecting the single principle responsibility.	none
20	All outputs and inputs should be handled via the CMD window	yes	none	none
21	Undo moves should be available for the medium and hard game modes	yes	none	none
22	A timer will be set for the other two modes (5sec=hard, 10 sec=medium) and none for the easy mode.	Yes	For the timer, 10 and 5 seconds is not enough time for the user because some of that time is being taken for the program to load and process, so for the updated timer requirement (medium=20 seconds, hard=10)	none

23	The game will have 3 different game modes: easy/medium/hard	yes	none	None
24	The swiftbot will move along the squares and return to starting position			
25	Calibration of the swiftbot should involve checking if the robot moves within 25x25 board squares without crossing over to another square for every move and should keep track of its current location and orientation before moving to the next square for swiftbot's turn.	Multiple tests are done on multiple surfaces (smooth wooden table, tablecloth and on paper). Overall, the tablecloth for me is the closest and more accurate of the tests because it had a smoother surface and that testing on paper would make the paper move, since taping it will also cause subtle friction.	The distance the swiftbot travels per millisecond along squares is originally set to 1000ms, however, the final test, I decided 1000ms would be suitable since multiple tests is closer to that value anyway, including the mean.	None

EVALUATION (BLACK BOX TESTING)

1) FUNCTIONAL REQUIREMENTS TESTING

Test case ID	Test case description	Requirement number	Test data/actions taken	Expected value/behaviour	Observed value/behaviour	Status (pass/Fail)	comments
1	The program starts with a welcome message and displays game rules	2.3) a)	Run the program via ssh with the swiftbot API together	The output should first display a welcome message and then a summary of the game rules.	the output displays both the welcome message and game rules as intended. [see screenshot below]	Pass	
2	The program should start the registration sequence, prompt the user for a username and generate a random name for the swiftbot that can be configured	2.3) b) c) d)	Press the 'A' button key on swiftbot (not ENTER key) 'ALBERT67' 'Albert 67' 'Albert67' 'ALBERTGUO2523488#' '[EMPTY]'	1) User should see the output like 'Enter your player's name (alphanumeric, 1-15 chars): >' upon entering'. Prompting the user to enter a username. 2) program should randomly generate a name for the swiftbot.	- The program asks for the user to enter a username within the defined range and format - the program generates a name for the swiftbot randomly - they then ask if the user wanted	Pass	

			'A'	3) the swiftbot name can be given the option to be manually configured.	to change the robot's username [see screenshots below]		
3	The program should roll a 6-sided dice pseudo randomly for both players.	2.4) a) b) c)	N/A	The program should display the dice results for two. If the dice roll is a tie, the program shall re-roll again until one output is greater than the other	-displays the dice roll results and assigns a starter the appropriate piece [see screenshot below]	Pass	
4	- The system will prompt the user to enter a move in the format '[row, col]' in the correct order - verify that the user input is numeric, in the right format and cannot make consecutive moves. - Any invalid inputs are rejected and re-prompted	2.5) a) b) c) g)	'2,2' '2, 2' '4,2' 'swe, www' '2 2'	The system will accept moves in the form a string format '[row, column]' include that comma in-between. It will reject any offending inputs, ranging from non-numerical ones to missing commas and invalid ranges It will also prevent consecutive moves, as this gives an unfair advantage	Largely works as intended, it accepts the format but rejects any out of bounds values. However, it also doesn't accept the brackets, just the values themselves The system doesn't display an error message for consecutive moves as the turn automatically moved on to the swiftbot. This is intended.	Pass	
5	The system will then check if the selected square based on the move is available. if not, it will notify the	2.5) d) e)	N/A	The system should reject any moves over a pre-existing player or swiftbot piece as that will lead to cheating. It will then display a message that the	The system displays '[ERROR] Square [2,2] occupied. Try again.' And asks for a re-prompt.	Pass	

	user for a re-prompt.			square is occupied and asking them to try again			
6	The system will mark a square an 'X' or 'O' piece on the chosen square if moves are valid and updates the board and displays move info.	2.5) f) h) l)	N/A	Updates the board with the new move and the move information that includes the player name, their game piece and the position they moved to.	Displays a new board with the updated move and outputs the player move details	Pass	
7	The swiftbot will randomly select its move within ranges and tells the user which square to move to and then blink in green 3 times upon arriving to the target square. Then displays the move details.	2.6) a) b) c) d) e)	N/A	the system will display which square the swiftbot will go to and will blink in green 3 times upon arriving. It should display which square it has moved to.	It displayed the swiftbot's username, play piece and the move it is moving to and displays the status of if it had already moved there	Pass	
8	The swiftbot will blink in green 3x before and after tracing an 'O' win, and blink in red 3x before and after tracing an 'X' win. For a draw, blink in blue	2.7) a) b) c)	N/A	For 'O' win: the swiftbot should blink its LEDs in green 3x before and after to trace a line. For X win: the swiftbot should blink in red 3x before and after drawing a line For draw: blink in blue 3x before	Without a physical board, we cannot make sure the distance the swiftbot travelled. But based on the observations, the swiftbot only blinked 3 times before and after it drew a line	Fail	

	3x before and after performing a spin			and after the move.	for both the user and swiftbot. The swiftbot did not spin once between blinking in blue 3x and after for draws.		
9	After each round ends, the program will record the results every time the following information (round number, playing piece, move positions, game result, winner name and current date and time. log file should stop writing after program terminates	2.8) a) d)	N/A	The program should record and write and append the results to the log file. The file should be available to open in winscp and should be downloadable as a text file.	The log file is created and its contents are there as intended but the formatting and spacing is a bit off and messy, overall, it meets the requirements.	Pass	
10	The scoreboard should be updated per round, and the players should have their score incremented every time one of them wins. It should be displayed as well.	2.8) b) c) e)	N/A	Users should see the displayed scoreboard upon each round, cleanly formatted with sufficient spacing, player scores should increment upon each round every time.	Round scoreboard is displayed and the formatting is very consistent with even ascii escape codes to colour code outputs, and the round number is incremented along with the score.	Pass	
11	Program tells user to press 'X' to quit or 'Y' to continue the	2.9) a) b)	Press 'Y' or 'X' on swiftbot	If X: the program should terminate peacefully without	The buttons works as intended	Pass	

	game on swiftbot.			exceptions and errors If Y: resets the program and starts again			
--	-------------------	--	--	--	--	--	--

2) NON-FUNCTIONAL REQUIREMENTS TESTING

<u>Test case ID</u>	<u>Test case description</u>	<u>Requirement number</u>	<u>Test data/actions taken</u>	<u>Expected value/behaviour</u>	<u>Observed value/behaviour</u>	<u>Status (pass/fail)</u>	<u>comments</u>
1	Messages/outputs should be clear, concise and detailed without being overly complex or non-understandable.	3.1) c)	N/A	The messages should be consistent and labelled with signs such as 'ERROR', 'WARNING', 'PROCESS' for example.	The messages displayed are correct, clear and detailed but they are colour coded, and the messages are consistent	Pass	
2	The UI and board formatting, design and structure should be clear, simple and organized and should only be displayed/rendered correctly in the CMD interface	3.1) a) b) d)	Add spacing (new print statement lines in eclipse) Between each output to increase clarity and attention.	The output displayed should be clear, with ascii color-coding, adequate spacing and orderly.	The UI screen is reasonably organized and uses consistent colour schemes and labelling to categorize outputs to avoid confusion. the board isn't displayed for the first time after the system displays the dice roll results but displays the board after if either player moves.	Pass	The board occasionally displays empty squares in some runs and fail to render correctly.
3	Errors and exceptions should be consistently handled by the program safely and the appropriate error message should be displayed that matches the issue found, including user inputs without terminate the program.	3.1) c) 3.3) a) b) 3.8) e)	to enter a username: 'albert 67' 'ALBERT67' 'albertalbert guo67' 'albert67' Enter user move: '3,4' '1,2' '45'	Error handling should be consistent, implemented adequately and should be displayed whether the user enters an invalid input. If a user enters an invalid input, they should be	For the username entry and user moves, input validation and error handling has been successfully handled and displays the correct outputs. However, when it comes to changing the username of the swiftbot, any invalid	Fail	

	Ensures that all input validation is consistent and tidy.		'2 2' Changing the swiftbot's generated username (yes/no) : 'r4' 'y3s' 'No' 'Yes' 'NO' 'YES'	re-prompted again rather than terminating the program.	inputs other than 'yes' or 'no' is accepted meaning error handling it non-existent here and when the user choosing the difficulty of the game. Other parts are not consistent applied too.		
4	The program should have code that should be easily understandable and maintainable such as using code comments, modularity and refactoring. It should apply these principles consistently (not arbitrarily) across the program's architecture.	3.5) a) b)	N/A	the program should have distinct linked classes and methods with comments explain their purpose, functions and refactored to make it easier to maintain.	The program has two classes, with each method doing at least 1 job. However, 1 or more classes should be created to encompass to accommodate the distinct helper and utility methods	Fail	
5	The system should perform as expected by calculating moves, positions and update boards at least 3 seconds the maximum with minimal performance delays.	3.2) a)	N/A	The system should wait no longer than 3 seconds than perform the essential functions and should respond upon input or directions.	The program responded near-instantaneously upon user events or performing the required tasks, such as blinking the swiftbot LEDs	Pass	

3) ADDITIONAL FUNCTIONALITIES TESTING

Test case ID	Test case description	Requirement number	Test data/actions taken	Expected value/behaviour	Observed value/behaviour	Status (pass/Fail)	comments
1	Undo moves that the user last made and has a limit cap how many they should have depending on game mode. The system should prevent user from undo more after it run out.	4.1)	'un3o' 'undo' 'Undo'	The program should accept the exact keyword 'undo' in correct format (case sensitive). The undo function prompt was physically integrated into	Entering 'undo' will make prompt the user to revise their choices, if not, then they can proceed with them, and the program should revert their move successfully in	Pass	

				the username entry and any errors made will be rejected.	the displayed, updated board		
2	A timer will be implemented for the medium and hard modes, which decreases with difficulty. User should enter before runs out.	4.2)	N/A	There should be a time countdown that the numbers decrements on one line until the move becomes invalid.	For the medium and hard modes, the timer decrements as intended but it decrements on each new line without overwriting the previous line.	Pass	The timer actually counts down on a new line every time, living little time or room for the user to enter their moves.
3	The game will ask the user for a game difficulty during registration and the game should adjust the undo moves and timer for each mode.	4.3)	'easy' 'medium' 'hard' 'e3sy' 'Midium' 'haart'	The program should ask for a difficulty and applies input validation on these inputs	The program did ask the user for a difficulty, but it also accepts other inputs other than these three without input validation.	Pass	

FULLY UPDATED SOURCE CODE LISTINGS

```

import java.io.*;
import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;
import java.util.*;
import swiftbot.Button;
import swiftbot.SwiftBotAPI;
/**
 * @author Albert Guo
 * Last updated: 15:25, 07/03/2026
 * PURPOSE OF PROGRAM: this program commands the Swiftbot to perform a 1v1 game with the user
 * in a game of noughts and crosses.
 * FUNCTIONALITIES: it handles user input via swiftbot buttons, robot movement and handles general game logic
 */

/**
 * the class for the overall program. this class contains, including the GameController class, which has all the game logic, swiftbot
 * movements, button event handling
 * and the ASCII colour escape codes to colour categories of outputs for organization and clarity
 */
public class Noughts_and_Crosses {
    static final String RESET = "\u001B[0m";
    static final String RED = "\u001B[31m";
    static final String BLUE = "\u001B[34m";
    static final String GREEN = "\u001B[32m";
    static final String YELLOW = "\u001B[33m";
    static final String ORANGE = "\u001B[38;2;255;165;0m";
    static final String PURPLE = "\u001B[35m";
    static final String CYAN = "\u001B[36m";
    static final String MAGENTA = "\u001B[35m";
    static final String LIGHT_GREEN = "\u001B[38;2;144;238;144m";
    static final String LIGHT_BLUE = "\u001B[94m";
    static final String LIGHT_PURPLE = "\u001B[95m";
    /**
     * main method that calls and creates a new instance of the static class 'GameController'
     * and assigns it to the variable 'game'.
     * then calls the 'run' method for that instance and acts as the main entry point to the program
     */
    public static void main(String[] args) {
        CoreGameLogic game = new CoreGameLogic();
        game.run();
    }
    /**
     * this is the main class that handles the bulk of the game's central logic
     * the @private access modifier are there to show that these declared variables belong
     * only to CoreGameLogic and can only be accessed within that class itself. (Encapsulation)
     */
    static class CoreGameLogic {

```

```

/*provides access to all swiftbot hardware functionalities in the swiftbotAPI*/
private SwiftBotAPI api;

/*a string 2D array called 'InitializeBoard' representing a 3x3 gameboard for storing X/O positions*/
private String[][] InitializeBoard = new String[3][3];

/*represents the username entered by the player*/
private String UserName = "";

/*the randomly generated username assigned to the swiftbot*/
private String SwiftbotName = "";

/*randomly assigns game pieces (X or O) based on dice roll and assigns to the user and swiftbot*/
private String UserPiece = "";
private String SwiftbotPiece = "";

/*the first turn the player takes after winning the dice roll*/
private String CurrentTurn = "";

/*increments the score number every time the player/swiftbot wins for each round*/
private int UserScore = 0;
private int SwiftbotScore = 0;

/*assigns the results of the dice roll to these variables*/
private int UserRoll = 0;
private int SwiftbotRoll = 0;

/*increments the round number every time a round is completed*/
private int RoundNumber = 0;

/*reverses a occupied square back to its original state via undoing moves*/
private int LastUserRow = -1;
private int LastUserCol = -1;

/*three modes of difficulty: easy/medium/hard*/
private String difficulty = "";

/*reverse moves to go back to the original state of the board*/
private int UndoMove;

/*time limit for moves for only the medium and hard modes*/
private int CountdownTimer;

/*the number of undos the user has left*/
private int UndoMovesRemaining;

/*new scanner instance for reading and responding to user inputs from keyboard*/
private Scanner in = new Scanner(System.in);

/*new random instance to roll a dice to determine turns*/
private Random random = new Random();

/*sets the conditions for swiftbot connection/availability to be false to ensure
that the program doesn't mistakenly thinks that the swiftbot is connected*/
private boolean SwiftbotOn = false;

/*an array list of strings containing the results from each round and assigns them to an
array list as string*/
private List<String> LogRecord = new ArrayList<>();

/**
 * detects if the swiftbot is connected and detects any exceptions that may cause it to do so otherwise
 */
public CoreGameLogic() {
    try {
        api = SwiftBotAPI.INSTANCE; /*an instance of swiftbotAPI that checks if the swiftbot is available*/
        SwiftbotOn = true;
        System.out.println(YELLOW + "[SUCCESS] The swiftBot has been connected successfully." + RESET);
    } catch (Exception e) {
        System.out.println(ORANGE + "[WARNING] SwiftBot isn't detected or connected. please try again or use keyboard inputs" + RESET);
        SwiftbotOn = false;
    }
}

/**
 * displays the game rules, handles button even handling, registers the user and inputting, assigning and generating
 * a username for both user and swiftbot
 * implements input validation rules for usernames by regex
 */
public void initAndRegister() {
    System.out.println("=====");
    System.out.println("----- NOUGHTS & CROSSES -----");
    System.out.println("=====");
    System.out.println("Welcome! Play Noughts & Crosses 1v1 against SwiftBot.");
    System.out.println();
    System.out.println("----- SUMMARY OF GAME RULES: -----");
    System.out.println("1) Enter moves in the format [row,column]. these should be in integer form only.e.g, (1,3) with no spaces in-between. ");
    System.out.println();
    System.out.println("2) there will be 3 modes of difficulty (easy/medium/hard)");
    System.out.println();
    System.out.println("3) there will be a time limit applied only to the medium and hard modes. the easy mode will have no such limit. ");
    System.out.println();
    System.out.println("4) Use 'undo' to revert your last move (if available)");
    System.out.println();
    System.out.println("5) user undos vary by difficulty. for instance, hard mode (no undo), medium mode (1 undo) and easy mode (2 undos) "

```

```

+ "these undos are here in case the user wants to revise back their choices ");
System.out.println();
System.out.println(" 6) the user only wins the game when he/she attains 3 'X' or 'O' consecutively. whether horizontal, vertical or
diagonal");
System.out.println();
System.out.println(" 7) if its a tie, the swiftbot will blink blue 3x before and after spinning once");
System.out.println("-----");
System.out.print("\n[INPUT] Please press button A on the swiftbot to begin... ");

if (SwiftbotOn) {

    try {
        api.setButtonLight(Button.A, true);
        boolean [] pressed = { false };
        api.enableButton(Button.A, () -> {
            pressed[0] = true;
            api.disableButton(Button.A);
            api.setButtonLight(Button.A, false);});

        while (!pressed[0]) {
            Thread.sleep(100);
        }
    } catch (Exception e) {in.nextLine();}
} else {
    in.nextLine();
}

while (true) {
    System.out.println();
    System.out.println(MAGENTA + "[PROCESS] starting registration...." + RESET);
    System.out.println();
    System.out.println("-----");
    System.out.print("[INPUT] Please enter a username (alphanumeric, 1-15 chars maximum):\n> ");
    System.out.println(LIGHT_GREEN);
    System.out.println("-----");

    UserName = in.nextLine().trim();
    if (UserName.matches("[a-z0-9]{1,15}$"))
        break;

    System.out.println();
    System.out.println(RED + "*** [ERROR] Invalid username. it cannot be empty, contain spaces, capitals and maximum
is 15 characters long. ***" + RESET);
    System.out.println();
}
System.out.println(YELLOW + "[SUCCESS] Username set: " + RESET + UserName);
System.out.println();
System.out.println(MAGENTA + "[PROCESS] rolling the dice now...." + RESET);
System.out.println();

String random_5_letters = "";
int length = 5;

for (int i = 0; i < length; i++) {
    int random = (int) (Math.random() * 26);

    char randomChar = (char) ('a' + random);
    random_5_letters += randomChar;
}
SwiftbotName = random_5_letters + (1000 + new Random().nextInt(1000));
System.out.println();
System.out.println(MAGENTA + "[PROCESS] Generated name for the swiftbot: \"\" + SwiftbotName + "\"\" + RESET);
System.out.println();
System.out.print("[INPUT] do you want to change SwiftBot name? (yes/no): \n> ");
String yn = in.nextLine().trim();

if (yn.equalsIgnoreCase("yes")) {
    System.out.println();
    System.out.print("[INPUT] Enter SwiftBot new name: \n> ");
    SwiftbotName = in.nextLine().trim();
}
else if (yn.equalsIgnoreCase("no")) {
    System.out.println();
    System.out.println(YELLOW + "[SUCCESS] your swiftbot name has been set: " + RESET + SwiftbotName);
}
else {
    System.out.println();
    System.out.println("*** [ERROR] Please enter 'yes' for change or 'no' for staying the same ***");
    SwiftbotName = in.nextLine().trim();
}

}

/**
 * rolls a random integer from 1-6 and if either the user/swiftbot goes first if one of them wins
 * if its a draw, the dice re-rolls until one gets a higher score
 * displays which goes first based on results
 */
public void rollDice() {
    System.out.println(MAGENTA + "\n[PROCESS] rolling dice now...." + RESET);

    do {
        UserRoll = random.nextInt(6) + 1;
        SwiftbotRoll = random.nextInt(6) + 1;

        System.out.println("***** -- OUTCOME -- *****");
        System.out.println("[\" + UserName + "]" + " rolled a: " + UserRoll);
        System.out.println(SwiftbotName + " rolled a: " + SwiftbotRoll);
        System.out.println("*****");
    }
}

```

```

        if (UserRoll == SwiftbotRoll) {
            System.out.println(MAGENTA + "[PROCESS] the dice rolled is a tie. re-rolling....." + RESET);
        }
        while (UserRoll == SwiftbotRoll);
        if (UserRoll > SwiftbotRoll) {
            UserPiece = "O";
            SwiftbotPiece = "X";
            CurrentTurn = "user";
        } else {
            SwiftbotPiece = "O";
            UserPiece = "X";
            CurrentTurn = "bot";
        }
        String StarterName = CurrentTurn.equals("user") ? UserName : SwiftbotName;
        String StarterPiece = CurrentTurn.equals("user") ? UserPiece : SwiftbotPiece;

        System.out.println(PURPLE + "\n[RESULT] " + StarterName + " goes first with piece: " + StarterPiece + RESET);
        System.out.println(GREEN + "----- GAMEBOARD (EMPTY) -----" + RESET);
        CalibrationHandling.DisplayBoard(InitializeBoard);
    }
    /**
     * user enters move in format [row,column] and implements input validation to check for format
     * if format is invalid, prompts user to re-enter otherwise accepts the move.
     * an option to undo move only if the user had a previous move before, then calls the undo method
     * handles the countdown time for the medium and hard modes,
     * @return true if a move is successfully accepted and processed
     */
    public boolean userTurn() {
        boolean[] timedOut = { false };

        if (CountdownTimer > 0) {
            Thread timerThread = new Thread(() -> {
                try {
                    for (int i = CountdownTimer; i > 0; i--) {
                        System.out.print(YELLOW + "\r[TIMER] " + BLUE + i + RESET + " seconds remaining... " + RESET);
                        Thread.sleep(1000);
                    }
                    timedOut[0] = true;
                    System.out.println();
                } catch (InterruptedException e) {
                }
            });
            timerThread.setDaemon(true);
            timerThread.start();
        }

        while (true) {
            System.out.println();
            System.out.println("-----");
            System.out.println();
            System.out.println(BLUE + "[INFO] Your turn is up now: " + UserName + " [" + UserPiece + "]" + RESET);
            System.out.println();
            System.out.println(BLUE + "[INFO] Undos left: " + UndoMovesRemaining + " Time limit: " + CountdownTimer + "s" + RESET);
            System.out.println();
            System.out.print("[INPUT] Please enter move in the format (row,column) with a comma please or enter 'undo' to undo moves:\n");
            System.out.print(LIGHT_GREEN);

            String input = in.nextLine().trim();

            if (timedOut[0]) {
                LogRecord.add("FORFEIT," + UserName + " , " + LocalDateTime.now());
                return false;
            }
            if (input.isEmpty()) {
                continue;
            }
            if (input.equalsIgnoreCase("undo")) {
                handleUndoMoves();
                continue;
            }
            String[] parts = input.split(",");
            if (parts.length != 2) {
                System.out.println(RED + "*** [ERROR] Invalid format. Use row,column e.g. 2,3 ***" + RESET);
                continue;
            }
            int row, column;
            try {
                row = Integer.parseInt(parts[0].trim());
                column = Integer.parseInt(parts[1].trim());
            } catch (NumberFormatException e) {
                System.out.println(RED + "*** [ERROR] Numbers only please. ***" + RESET);
                continue;
            }
            if (row < 1 || row > 3 || column < 1 || column > 3) {
                System.out.println(RED + "*** [ERROR] Row and column must each be between 1 and 3. ***" + RESET);
                continue;
            }
            if (!InitializeBoard[row - 1][column - 1].equals(" ")) {
                System.out.println(RED + "*** [ERROR] Square [" + row + ", " + column + "] is already taken. Try again. ***" + RESET);
                continue;
            }
            InitializeBoard[row - 1][column - 1] = UserPiece;
            LastUserRow = row;
            LastUserCol = column;
            System.out.println();
            System.out.println(PURPLE + "[RESULT] " + UserName + " [" + UserPiece + "]" + "moved to [" + row + ", " + column + "]" + RESET);
        }
    }

```

```

        LogRecord.add(Username + "," + UserPiece + "," + row + "," + column + "," + LocalDateTime.now());
        System.out.println();
        System.out.println(GREEN + "----- GAMEBOARD -----" + RESET);
        CalibrationHandling.DisplayBoard(InitializeBoard);
        return true;
    }

    /**
     * calls the handle undo moves method and only reverts the users move if
     * they had at least 1 move before, otherwise the undo will fail.
     * the user will need to confirm their options
     */
    public void handleUndoMoves() {

        if (UndoMovesRemaining <= 0) {
            System.out.println(ORANGE + "--- [WARNING] No undos remaining. ---" + RESET);
            return;
        }
        if (LastUserRow == -1) {
            System.out.println(ORANGE + "--- [WARNING] No move to undo yet. ---" + RESET);
            return;
        }
        System.out.print("[INPUT] Confirm undo? (yes/no):\n> ");
        String conf = in.nextLine().trim();
        if (!conf.equalsIgnoreCase("yes")) {
            return;
        }
        InitializeBoard[LastUserRow - 1][LastUserCol - 1] = " "; /*resets the user's last move to return square
                                                                    to original state*/
        UndoMovesRemaining--; /*deducts the number of moves available*/
        LastUserRow = -1;
        LastUserCol = -1;
        System.out.println(YELLOW + "[SUCCESS] Move undone. Undos left: " + UndoMovesRemaining + RESET);
        System.out.println();
        System.out.println(GREEN + "----- GAMEBOARD -----" + RESET);
        CalibrationHandling.DisplayBoard(InitializeBoard);

    }
    /**
     * swiftbot will randomly select and go to the designated square and will tell
     * the user which square via [row,col] it will move to
     */
    public void SwiftBotTurn() {

        List<int[]> emptySquares = new ArrayList<>();
        for (int r = 1; r <= 3; r++) {
            for (int c = 1; c <= 3; c++) {
                if (InitializeBoard[r - 1][c - 1].equals(" ")) {
                    emptySquares.add(new int[]{ r, c });
                }
            }
        }
        if (emptySquares.isEmpty()) {
            return;
        }
        int[] chosen = emptySquares.get(random.nextInt(emptySquares.size()));
        int r = chosen[0];
        int c = chosen[1];
        System.out.println(BLUE + "\n[INFO] " + SwiftbotName + RESET + "[" + SwiftbotPiece + "]" + " will move to " + "[" + r + "," + c + "]" +
            RESET);
        if (SwiftbotOn) {
            try {
                int MilliSecPerCell = 1000; /*the time value that the swiftbot takes to move per cell at 100% its speed*/
                api.move(100, 100, r * MilliSecPerCell);

                Thread.sleep(300);
                api.move(100, -100, c * MilliSecPerCell);
                Thread.sleep(300);

                CalibrationHandling.BlinkLEDLights(api, SwiftbotOn, "GREEN", 3);

                api.move(-100, 100, c * MilliSecPerCell);
                Thread.sleep(300);
                api.move(-100, -100, r * MilliSecPerCell);
                Thread.sleep(300);

            } catch (Exception e) {
                System.out.println(RED + "*** [ERROR] SwiftBot motion error: " + e.getMessage() + " ***" + RESET);
            }
        }
        InitializeBoard[r - 1][c - 1] = SwiftbotPiece;
        System.out.println();
        System.out.println(PURPLE + "[RESULT] " + SwiftbotName + " [" + SwiftbotPiece + "] moved to [" + r + "," + c + "]" + RESET);
        LogRecord.add(SwiftbotName + "," + SwiftbotPiece + "," + r + "," + c + "," + LocalDateTime.now());
        System.out.println();
        System.out.println(GREEN + "----- GAMEBOARD -----" + RESET);
        CalibrationHandling.DisplayBoard(InitializeBoard); /*calls the class method 'DisplayBoard' from
                                                            the class 'CalibrationHandling' without
                                                            showing true complexity (Abstraction) */
    }
    /**
     * evaluates the whole board based on if the user/swiftbot gets 3 in a row vertically, horizontally or diagonally
     * @return null if one of the 2 players get three consecutive squares in a row
     * @return "DRAW" if both players don't get 3 in a row
     */
    public String CheckWinOrDraw() {

        for (int r = 0; r < 3; r++) {

```

```

        if (!InitializeBoard[r][0].equals(" ")
            && InitializeBoard[r][0].equals(InitializeBoard[r][1])
            && InitializeBoard[r][1].equals(InitializeBoard[r][2])) {
            return InitializeBoard[r][0];
        }
    }
    for (int c = 0; c < 3; c++) {
        if (!InitializeBoard[0][c].equals(" ")
            && InitializeBoard[0][c].equals(InitializeBoard[1][c])
            && InitializeBoard[1][c].equals(InitializeBoard[2][c])) {
            return InitializeBoard[0][c];
        }
    }
    if (!InitializeBoard[0][0].equals(" ")
        && InitializeBoard[0][0].equals(InitializeBoard[1][1])
        && InitializeBoard[1][1].equals(InitializeBoard[2][2])) {
        return InitializeBoard[0][0];
    }
    if (!InitializeBoard[0][2].equals(" ")
        && InitializeBoard[0][2].equals(InitializeBoard[1][1])
        && InitializeBoard[1][1].equals(InitializeBoard[2][0])) {
        return InitializeBoard[0][2];
    }
    for (int r = 0; r < 3; r++) {
        for (int c = 0; c < 3; c++) {
            if (InitializeBoard[r][c].equals(" ")) {
                return null;
            }
        }
    }
    return "DRAW";
}
/**
 * this method, based on the completed evaluation in the CheckWinOrDraw method, decides if the user/swiftbot ends
 * a win or a draw, and blinks the appropriate LED lights upon printing out the message
 * @param result - the string that is compared against the results (WIN/DRAW) conditions from this method
 * and if the string matches one of the 3 conditions, it returns that specific outcome.
 */
public void RoundOutcome(String result) {
    if (result.equals("DRAW")) {
        System.out.println(BLUE + "\n***** IT'S A DRAW! ***** + RESET);
        System.out.println();
        System.out.println();
        System.out.println("*****");
        System.out.println();
        System.out.println(    UserName + " : " + UserScore);
        System.out.println();
        System.out.println(    SwiftbotName + " : " + SwiftbotScore);
        System.out.println();

        if (SwiftbotOn) {
            System.out.println();
            System.out.println(BLUE + "[ACTION] blinking in blue 3 times.... " + RESET);
            System.out.println();
            CalibrationHandling.BlinkLEDLights(api, SwiftbotOn, "BLUE", 3);
            System.out.println();
            System.out.println(BLUE + "[MOTION] spinning.... " + RESET);
            System.out.println();
            CalibrationHandling.Spin(api, SwiftbotOn);
            System.out.println();
            System.out.println(BLUE + "[ACTION] blinking in blue 3 times.... " + RESET);
            System.out.println();
            CalibrationHandling.BlinkLEDLights(api, SwiftbotOn, "BLUE", 3);
        }

    } else if (result.equals(UserPiece)) {
        System.out.println("\n***** " + GREEN + UserName + RESET + " WINS! *****");
        UserScore++;

        if (SwiftbotOn) {
            System.out.println();
            System.out.println(GREEN + "[ACTION] blinking in green 3 times.... " + RESET);
            System.out.println();
            CalibrationHandling.BlinkLEDLights(api, SwiftbotOn, "GREEN", 3);
            System.out.println();
            System.out.println(GREEN + "[MOTION] Tracing winning line for: " + RESET + UserName);
            System.out.println();
            api.move(100, 100, 3000);
            System.out.println(GREEN + "[ACTION] blinking in green 3 times.... " + RESET);
            System.out.println();
            CalibrationHandling.BlinkLEDLights(api, SwiftbotOn, "GREEN", 3);
        }
    } else {
        System.out.println("\n***** " + RED + SwiftbotName + RESET + " WINS! *****");
        SwiftbotScore++;
        if (SwiftbotOn) {
            System.out.println();
            System.out.println(RED + "[ACTION] blinking in red 3 times.... " + RESET);
            System.out.println();
            CalibrationHandling.BlinkLEDLights(api, SwiftbotOn, "RED", 3);
            System.out.println();
            System.out.println(RED + "[MOTION] Tracing winning line for: " + RESET + SwiftbotName);
            System.out.println();
            api.move(100, 100, 3000);
            System.out.println(RED + "[ACTION] blinking in red 3 times.... " + RESET);
            System.out.println();
            CalibrationHandling.BlinkLEDLights(api, SwiftbotOn, "RED", 3);
        }
    }
}

```

```

    }

    /**
     * asks the user to press or enter X to quit or Y to continue/play again.
     * button light is one unless the swiftbot is connected and only turns off if
     * either button is pressed
     * displays message on which button is pressed
     * @return 'Y' for playing again
     */
    public boolean PlayAgainOrQuit() {
        System.out.println("\n-----");
        System.out.println("[INPUT] Please press button Y to play again or X to quit (or type (X/Y)) and press Enter:\n> ");
        System.out.println("-----");

        char choice = '?';

        if (SwiftbotOn) {
            try {
                api.setButtonLight(Button.Y, true);
                api.setButtonLight(Button.X, true);

                boolean[] pressed = { false };
                char[] result = { '?' };

                api.enableButton(Button.Y, () -> {
                    if (!pressed[0]) {
                        result[0] = 'Y';
                        pressed[0] = true;
                    }
                });
                api.enableButton(Button.X, () -> {
                    if (!pressed[0]) {
                        result[0] = 'X';
                        pressed[0] = true;
                    }
                });
                while (!pressed[0]) {
                    Thread.sleep(100);
                }
                api.disableAllButtons();
                api.setButtonLight(Button.Y, false);
                api.setButtonLight(Button.X, false);
                choice = result[0];
                System.out.println();
                System.out.println(CYAN + "[ACTION] Button " + choice + " pressed." + RESET);
                System.out.println();

            } catch (Exception e) {
                System.out.print("[INPUT] Enter Y or X:\n> ");
                String s = in.nextLine().trim().toUpperCase();
                choice = s.isEmpty() ? '?' : s.charAt(0);
            }
            } else {
                System.out.print("[INPUT] Enter Y or X:\n> ");
                String s = in.nextLine().trim().toUpperCase();
                choice = s.isEmpty() ? '?' : s.charAt(0);
            }
        }
        return choice == 'Y';
    }

    /**
     * runs the general program and calls other methods
     * handles the selection of the games difficulty and then assigns them number of undos and time limit for
     * each move (easy mode has no time limit)
     */
    public void run() {
        initAndRegister();

        while (true) {
            System.out.println();
            System.out.print("[INPUT] Choose difficulty (easy / medium / hard):\n> ");
            String d = in.nextLine().trim().toLowerCase();

            if (d.equals("easy")) {
                difficulty = "easy";
                UndoMove = 2;
                CountdownTimer = 0;
                break;
            } else if (d.equals("medium")) {
                difficulty = "medium";
                UndoMove = 1;
                CountdownTimer = 10;
                break;
            } else if (d.equals("hard")) {
                difficulty = "hard";
                UndoMove = 0;
                CountdownTimer = 5;
                break;
            } else {
                System.out.println(RED + "**** [ERROR] invalid input. please enter (easy/medium/hard) ****" + RESET);
            }
        }

        System.out.println(BLUE + "[INFO] Difficulty selected: " + difficulty + " | Undo moves allowed attempts: " + UndoMove + " | Timer: " +
            CountdownTimer + "s" + RESET);

        boolean keepPlaying = true;
        while (keepPlaying) {

```

```

rollDice();

for (int r = 0; r < 3; r++) {
    for (int c = 0; c < 3; c++) {
        InitializeBoard[r][c] = " "; /*maintains the board as long as the game is still playing*/
    }
}
UndoMovesRemaining = UndoMove; /*deducts the number of undos everytime the user uses it*/
LastUserRow = -1;
LastUserCol = -1;
boolean roundOver = false;
while (!roundOver) {
    if (CurrentTurn.equals("user")) {
        userTurn();
        CurrentTurn = "bot";
    } else {
        SwiftBotTurn();
        CurrentTurn = "user";
    }
    String result = CheckWinOrDraw();

    if (result != null) {
        RoundOutcome(result);
        RoundNumber++;
        log.SaveLogAndDisplayScore(result, RoundNumber, UserName, SwiftbotName, UserScore, SwiftbotScore, UserPiece, SwiftbotPiece, LogRecord);
        roundOver = true;
    }
}
keepPlaying = PlayAgainOrQuit();
}
log.LogFileRecord(LogRecord);
System.out.println();
System.out.println(MAGENTA + "[PROCESS] Program terminating....." + RESET);
System.out.println();
System.out.println(YELLOW + "----- SEE YOU LATER! -----" + RESET);
}

/**
 * this class contains and handles the core calibration and movement logic for the Swiftbot to perform
 * and that other methods from the main class calls upon
 */
static class CalibrationHandling {
    /**
     * blinks the appropriate LED lights of either GREEN, RED or BLUE upon round conditions
     * @param LEDcolor is the colour that is activated when certain conditions are met
     * @param NumTimes is the number of times that colour is blinked, in this case 3x before and after
     * catches any exceptions that causes the blinking to fail, and displays an error
     */
    public static void BlinkLEDlights(SwiftBotAPI api, boolean SwiftbotOn, String LEDcolor, int NumTimes) {
        if (!SwiftbotOn) {
            System.out.println(GREEN + "[MOTION] swiftbot will blink " + LEDcolor + " x" + NumTimes + RESET);
            return;
        }
        int[] rgb;
        if (LEDcolor.equals("GREEN"))
            rgb = new int[]{ 0, 255, 0 };
        else if (LEDcolor.equals("RED"))
            rgb = new int[]{ 255, 0, 0 };
        else if (LEDcolor.equals("BLUE"))
            rgb = new int[]{ 0, 0, 255 };
        else
            rgb = new int[]{ 255, 255, 0 };

        try {
            for (int i = 0; i < NumTimes; i++) {
                api.fillUnderlights(rgb);
                Thread.sleep(300);
                api.disableUnderlights();
                Thread.sleep(300);
            }
        } catch (Exception e) {
            System.out.println(RED + "**** [ERROR] Blink failed: " + e.getMessage() + " ****" + RESET);
        }
    }

    /**
     * if the swiftbot is not disconnected or unavailable, then it will spin once upon meeting certain conditions
     * Spins at 100% the robot speed both forwards and backwards and runs for 600 milliseconds (0.6 seconds)
     * catches any exceptions and prints out that error
     */
    public static void Spin(SwiftBotAPI api, boolean SwiftbotOn) {
        if (!SwiftbotOn) {
            System.out.println(GREEN + "[MOTION] Spin once" + RESET);
            return;
        }
        try {
            api.move(100, -100, 600);
            Thread.sleep(100);
            api.move(100, -100, 600);
        } catch (Exception e) {
            System.out.println(RED + "**** [ERROR] Spin failed: " + e.getMessage() + " ****" + RESET);
        }
    }
}

```

```

        * displays the dimensions of the board labelling 'columns' and 'rows'
        *
        */
    public static void DisplayBoard(String[][] InitializeBoard) {
        System.out.println();
        System.out.println("Columns -> 1 2 3");
        System.out.println("Rows");
        for (int r = 0; r < 3; r++) {
            System.out.print("  " + (r + 1) + " ");
            for (int c = 0; c < 3; c++) {
                System.out.print("[ " + InitializeBoard[r][c] + " ");
            }
            System.out.println();
        }
        System.out.println();
    }
}

static class log {
    /**
     * the log saves the results of each round played
     * displays the score of each player, and the results of the round, ending in WIN or DRAW
     */
    public static void SaveLogAndDisplayScore(String result, int RoundNumber, String UserName, String SwiftbotName, int UserScore, int SwiftbotScore, String UserPiece, String SwiftbotPiece, List<String> LogRecord ) {
        String winner;
        if (result.equals("DRAW")) {
            winner = "DRAW";
        } else if (result.equals(UserPiece)) {
            winner = UserName;
        } else {
            winner = SwiftbotName;
        }
        String logLine = "Round " + RoundNumber + ", "
            + winner + ", "
            + result + ", "
            + LocalDateTime.now().toString();
        LogRecord.add(logLine);

        System.out.println("\n----- ROUND SUMMARY -----");
        System.out.println("Round number: " + LIGHT_BLUE + RoundNumber + RESET);
        if (result.equals("DRAW")) {
            System.out.println("Result: [DRAW]");
        } else {
            System.out.println("Result: " + LIGHT_PURPLE + "[" + winner + "]" + RESET + " WINS!");
        }
        System.out.println("\n----- SCOREBOARD -----");
        System.out.println(LIGHT_GREEN + UserName + RESET + ": " + LIGHT_BLUE + UserScore + RESET);
        System.out.println(LIGHT_BLUE + SwiftbotName + RESET + ": " + LIGHT_BLUE + SwiftbotScore + RESET);
        System.out.println("-----\n");
    }
}
/**
 * the log saves the results and and writes to the file for every round, and then shows the
 * log file path.
 */
    public static void LogFileRecord(List<String> LogRecord ) {
        String timestamp = DateTimeFormatter.ofPattern("yyyyMMdd_HHmms").format(LocalDateTime.now());
        String filename = "noughts_log_" + timestamp + ".txt";
        try {
            FileWriter fw = new FileWriter(filename, true);
            for (String entry : LogRecord) {
                fw.write(entry + "\n");
            }
            fw.close();
            System.out.println();
            System.out.println(YELLOW + "[SUCCESS] Log saved to: " + filename + RESET);
        } catch (IOException e) {
            System.out.println(RED + "*** [ERROR] Could not save log: " + e.getMessage() + " ***" + RESET);
        }
    }
}
}

```

REFERENCES

OpenAI ChatGPT 5.1 mini – *noughts and crosses*. This answer is generated by AI. Available on <https://chatgpt.com/c/693708ca-c110-832c-a638-a89209a49c35> (Generated 9 February 2025). See appendix A for further information.

Anthropic Claude Sonnet 4.6 Extended thinking - *Implementing button and LED controls in nought and crosses game*. Available on <https://claude.ai/chat/86c1d87a-5b6f-4069-bc01-1ccd6edfe840> (accessed: 23 February 2025). See appendix B for further information

Appendix A (ChatGPT)

A1. TOOL USED

OpenAI ChatGPT 5.1 mini

A2. PURPOSE OF USE

Used ChatGPT 5.1 mini to help me generate the full logic of each method and classes and the proto type of the software implementation (not full) where I would further improve on. This prototype would be based on following the method and class signatures that is based on the improved SRS and may change later due to improvements. This prototype serves as a window to further improve on my program.

A3. PROMPT PROVIDED

- 1) *'Break my assignment 2 project into 4 layers , layer 1 convert all flowcharts into method signatures and think about constraints and things used multiple flow charts add loads of notes too so I can understand layer 2 will be me editing the logic in each method signature layer 3 will be me fixing comments according to [proper software development principles and layer 4 will be last adding the swiftbot API instance and also adding that into the final program fir the bot'*
- 2) *'for layer 2, can you implement the logic for each subprocess and other methods'*

A4. FULL AI-GENERATED RESPONSE

To break down my implementation into layers:

'What you'll get now (all in this message):

- *Layer 1 — a complete list of method signatures (Java-style) converted from your flowcharts, plus constraints, shared utilities, and many detailed notes about reuse, race-conditions and test hooks.*
- *Layer 2 — precise instructions for editing/implementing logic inside every method (what to code, SRS references, edge cases).*
- *Layer 3 — guidance for comments, coding standards and software-engineering principles you must add (Javadoc templates, SOLID, testing strategy).*
- *Layer 4 — how to instantiate & integrate the SwiftBot API, wiring, safety, mock mode, and final program wiring steps.*

Work sequentially: implement Layer-2 logic for each method from Layer-1, then apply Layer-3 comment/clean-up rules, then wire real SwiftBot in Layer-4.'

LAYER 1 — Convert flowcharts → Method signatures (+ reusable components & constraints)

Below I list every top-level method that corresponds to your main flowchart and the 7 subprocesses. I also include helper/shared methods used across multiple flowcharts and notes about constraints and cross-use.

Guideline: paste these signatures into your GameController/Service classes; they're typed and throw exceptions where hardware or IO may fail.

LAYER 2 — Implement logic inside each method signature (step-by-step coding notes)

For each top-level method, I give a short checklist of exactly what to code inside, including error handling and test hooks. Do these in order; each step includes the SRS references you must satisfy.

initAndRegister() (SUBPROCESS 1)

What to implement:

Print welcome screen exactly like UI design (FR-01, NFR-09).

Wait for user start input (A) or hardware event. If hardware-enabled, prefer hardware event.

Prompt for player name; validate regex `^[A-Za-z0-9]{1,30}$`. Re-prompt on invalid input (FR-02). Provide error messages per SRS (single line, concise) (NFR-03).

Auto-generate SwiftBot name (e.g., Robo-xxx) and allow override prompt. Save both names (FR-03).

Save session_config defaults to SessionConfig for use by Difficulty setup.

Unit-test hooks: method should return Player object or set state in GameController for tests.

Edge cases:

Empty or whitespace-only input -> re-prompt.

Hardware button fires early -> skip manual input.

generateAndRollDice() (SUBPROCESS 2)

What to implement:

Random 1..6 for user & bot until not equal (tie loop) (FR-04).

Show outputs in CLI (two lines). Assign O to the higher roller per SRS; swap current accordingly.

Record dice results in in-memory event log (for traceability).

Unit tests: deterministic stub random (inject Random) for reproducible tests.

Edge cases:

If random generator seeds, allow overriding seed for tests.

userTurn() (SUBPROCESS 3)

What to implement (detailed):

Display board ASCII (board.asciiBoard()), display Undo quota and TimeLeft if timer configured (FR-09).

Start timer via TimerService.startTurnTimer(cfg.turnTimerSeconds, onTimeout->forfeit). Use ExecutorService + Future to read console input with timeout. (AF-02)

On input:

if "undo" -> call `handleUndo()` (AF-01) and return `UserTurnResult.UNDO`.

else parse `r,c` via `validateMoveInput()`; return `INVALID` and re-prompt on format issues. (FR-05)

Validate within range 1..3 and cell empty via `Board.isEmpty()` (FR-06). If invalid, show friendly message and re-prompt.

If valid, create `Move`, call `applyMoveAndUpdateUI(user, move)`. This should:

update board model

add move to current-round in-memory move list

print move summary within 1–2 seconds (NFR-04)

Cancel timer on successful move. Return `UserTurnResult.APPLIED`.

Test hooks:

Inject an `InputProvider` abstraction for unit tests to feed inputs (avoids blocking console).

Provide parameter to disable real timers in tests.

Edge cases:

Timer expiry while user typing -> must treat as forfeit and cancel future input (notify user).

Multiple invalid attempts consume time (document in SRS; pick policy and implement consistently).

swiftBotTurn() (SUBPROCESS 4)

What to implement:

Use `AIEngine` to choose move (random or strategic depending on `session_config`) (AF-03).

Announce chosen square in CLI before motion (FR-07).

Call `executeMotionSequence(targetRow, targetCol, meta)` which:

calls `swiftBot.isCalibrated()` (NFR-07)

computes motion plan (uses stored pose & seconds-per-25cm)

sends move commands via `swiftBot.moveTo()`

on arrival, calls `swiftBot.blink(GREEN,3)` and `swiftBot.returnToStart()` (FR-08, FR-19a/b)

logs motion telemetry

implements watchdog and safe-stop on timeouts (NFR-08)

After motion success, call `applyMoveAndUpdateUI(bot, move)` (same as for user) so board stays consistent; log the move.

Edge cases & notes:

If motion fails (IOException/MotionException), log it and continue with fallback (if appropriate).

For strategic AI, limit compute time to avoid blocking game excessively (e.g., 0.5–2s depending on difficulty).

checkWinDrawAndAct() (SUBPROCESS 5)

What to implement:

Call Board.checkWinner(outCoords) which returns Optional and winning coordinates (if any). (FR-10)

If winner:

If winner == user.piece -> swiftBot.blink(GREEN,3) then traceLine(outCoords) then blink(GREEN,3) (FR-22). Increment scoreboard.

Else -> blink(RED,3) trace, blink(RED,3) (FR-23).

If draw (board.isFull() and no winner): do swiftBot.spinOnce(); blink(BLUE,3) (FR-24).

Return RoundOutcome object for logging.

Important: Use executeMotionSequence for tracing to reuse calibration handling & watchdog.

Test hooks:

If hardware disabled, MockSwiftBot will print trace actions; use assertions to check method invocations.

appendRoundLogAndUpdateScoreboard(RoundOutcome outcome) (SUBPROCESS 6)

What to implement:

Build CSV line:

roundNumber,moveSequence(player|piece|row|col;...),result,winnerName,ISO_timestamp. (FR-11)

Append to in-memory logger LoggerService.append(..) and flush per policy (NFR-05) — flush immediately on quit, optionally flush per-round.

Update in-memory scoreboard and print scoreboard to CLI (FR-12).

On quit, persist scoreboard to disk.

Edge cases:

If flush fails due to IO, show friendly message and continue (NFR-08). Log technical details to debug file.

playAgainOrQuit() (SUBPROCESS 7)

What to implement:

Prompt user: Button Y or type 'Y' to play or Button X / 'X' to quit. (FR-13)

If quit:

Call LoggerService.flush(path) and print Path.

Persist scoreboard if required.

Cleanly shutdown MotionService & resources.

Return false.

If play again: reset Board, round state, re-initialize per-round lists and return true.

Utility: applyMoveAndUpdateUI(Player p, Move m)

What to implement:

Must be synchronized or atomic.

Steps:

board.applyMove(m).

logMove(m) (in-memory).

Print ASCII board and one-line move summary ([Name – Piece] moved to square [r,c]) within 1–2 seconds. (NFR-04)

Return success boolean.

Utility: executeMotionSequence(int r,int c, MotionMeta meta)

What to implement:

Check calibration swiftBot.isCalibrated() (NFR-07) -> refuse if not ok.

Compute motion plan using stored pose and secondsPer25cm param.

Send moveTo with timeout watchdog.

On arrival, call blink(GREEN,3) (or other color per meta).

Return to start.

Log telemetry and return status.

Extra utilities & safety

runCalibration() — perform calibration routine (measure seconds-per-25cm) and persist to config. Return CalibrationReport.

safeStopMotion() — sends emergency stop command if motion watchdog triggers.

serializeState() and deserializeState() — save session_config & scoreboard to persist between runs (optional but good for demo).

LAYER 3 — Comments, quality, tests, and software development principles

This layer is about how to comment, structure, and test — not what logic to put — so markers / future devs understand & maintain your work.

Commenting / Javadoc template (apply to every public method)

Use JavaDoc for each method and include these fields:

Single Responsibility: GameController orchestrates; Board, LoggerService, MotionService have single responsibilities.

Dependency Injection: pass SwiftBotInterface, LoggerService, ALEngine, TimerService into GameController constructor (makes testing easy).

Fail-safe defaults: if hardware fails, fallback to MockSwiftBot and inform user.

Encapsulation: do not expose internal board arrays; use methods.

Immutability where possible: Move objects immutable.

Small methods: each method should be $\leq$ ~80 lines, each doing one thing.

Naming: use meaningful names: applyMoveAndUpdateUI, executeMotionSequence, appendRoundLogAndUpdateScoreboard.

Error handling: catch exceptions at the appropriate level; log technical info to a debug log file; show friendly messages to user (NFR-08).

No magic numbers: define constants for timeouts, calibration tolerances, retry counts.

Logging & tracing conventions

Human readable logs: CSV for round logs. Separate debug log for stack traces.

Use ISO-8601 timestamps (Instant.now().toString()).

For each log line, include session_id and round_number.

Testing / Unit tests you must create

Create JUnit tests for:

Board.checkWinner() – all win/draw permutations.

validateMoveInput() – valid/invalid inputs.

undo logic: apply moves, undo, check board & log annotated.

ALEngine.chooseStrategicMove() – deterministic sandbox tests using seeded RNG or shallow depth.

LoggerService flush to temp file and read back lines.

MotionService with MockSwiftBot verifying calls.

Also add simple integration tests which run the GameController in --no-hw mode with an InputProvider to simulate rounds.

Code style & formatting

Adopt a style guide (Google Java Style or Sun).

Use consistent 4-space indent, line length $\leq$ 100.

Add @author and @version.

LAYER 4 — Add SwiftBot API instance + final wiring

This layer wires real hardware into your code and prepares final program for grading/demo.

SwiftBot API integration plan (safe + testable)

Abstraction: Keep SwiftBotInterface (already in Layer1). Real driver implements that (e.g., RaspberryPiSwiftBotDriver or ArduinoSwiftBotDriver). Only this class touches hardware I/O.

Dependency injection: In Main, create SwiftBotInterface driver = cfg.hardwareEnabled ? new RealDriver(...) : new MockSwiftBot(); then new GameController(cfg, driver, logger, aiEngine, timerService) — pass instances to constructor.

Calibration startup:

On program start (after registration), run calibrationReport = driver.runCalibration(); if not OK and hardware required, refuse to start and offer --no-hw or retry. Record calibration metrics. (SRS NFR-07)

Motion timeouts & watchdog:

Wrap hardware calls with Future and fixed timeout (e.g., 10s). If timeout -> call driver.safeStop() or driver.returnToStart() and log MotionFailure.

Hardware safety & emergency stop:

Provide a keyboard emergency stop input (e.g., typing !stop or physical E-STOP), that triggers safeStopMotion() which stops commands and sets hardwareEnabled=false temporarily.

Retries & graceful degradation:

On transient failures, attempt 1–2 retries. On repeated failure mark hardware unavailable and continue in --no-hw simulation mode. Log the change.

Wiring into final program

Main should parse CLI args (--no-hw, --ai=strategic, --logdir=...).

Instantiate LoggerService, AEngine, TimerService, SwiftBotInterface, GameController.

Call gameController.run().

On exit, ensure logger.flush() and driver.shutdown() (if present) are executed in finally block.

Final verification steps before submission

Run in --no-hw mode and perform a full manual playthrough of 1–2 rounds and verify:

Undo works (correct board & log annotated)

Timer works (timeout -> forfeit)

AI strategic moves appear reasonable

Logs are created and saved

Run hardware calibration test (if hardware present): verify `isCalibrated()` returns true and drift tolerance is reported.

Run `mvn test` or `gradle test` for the unit tests above; ensure passing.

Suggested implementation plan and priorities (practical)

Layer-1 code & DI skeleton — create interfaces & method signatures (1–2 hours).

Layer-2 implement core logic:

Implement `Board`, `applyMove`, `checkWinner`, `undo` (1–3 hours).

Implement `userTurn()` with `InputProvider` & timer abstractions (1–2 hours).

Implement `AIEngine` random (30 min) + minimax (2–3 hours).

Implement `LoggerService` (30–60 min).

Run manual tests in `--no-hw`, fix bugs (1–2 hours).

Layer-3 add comments & tests (2–3 hours).

Layer-4 hardware wiring — add `RealSwiftBot` driver and calibration routine (time depends on hardware).

Final polish: traceability matrix, add docs, produce screenshots for report.

Extra notes, gotchas & lots of tips (you asked for lots of notes)

Timer & console: pure Java cannot interrupt `System.in.read()` reliably on all consoles. Use an `InputProvider` abstraction so you can run unit tests, and for real CLI accept that timeouts may not cancel the typed input; treat timeouts as logical (ignore late input and move on) and display clear messages. Document this behavior in SRS.

Undo edge cases: If user issues `Undo` immediately after bot's move, define if undo reverts last move regardless of owner or only user moves. SRS earlier suggests “undo last move the player makes” — so limit undo to user-moves only; implement accordingly. Clarify and code the rule.

Logging format: use short CSV fields but avoid commas inside fields; prefer `;` or `|` as separators for move sequences, or quote fields properly.

Minimax performance: a full minimax with depth up to 9 is fine for 3×3; but add alpha-beta pruning for speed. Limit depth/time for hard vs easy.

Calibration storage: store `secondsPer25cm` in config file so motion planning can use it across sessions. Add a command `calibrate` to re-run.

Testing hardware interactions: use `MockSwiftBot` with counters to ensure `blink`/`move`/`trace` are called the expected number of times in tests.

Accessibility: duplicate color-labeled messages with text (e.g., `Blink GREEN`) for color-blind users (SRS NFR-09).

Traceability: in Javadoc for each method add SRS: FR-xx lines and include figure number for flowchart node — that helps markers.

Versioning & reproducibility: include LLM_provenance.txt with the prompt, model and timestamp in your submission appendix (assignment asked for it).

Final deliverable for you to copy/paste (short checklist)

[] Add the method signatures from Layer-1 into your codebase.

[] Implement logic per Layer-2 step list (start with Board & userTurn).

[] Add Javadoc & tests from Layer-3 for each method as you implement.

[] Wire SwiftBot per Layer-4 and test in --no-hw and hardware modes.

[] Add a one-page traceability matrix mapping FR/NFR/AF → method names → flowchart figure → UI screen.

[] Run unit/integration tests and fix any race/timer issues.

If you want, I can automatically generate:

(A) a Java file skeleton that contains all Layer-1 method signatures + Javadoc stubs referencing the SRS IDs (so you can immediately start Layer-2), or

(B) for one selected method (pick userTurn, swiftBotTurn, or executeMotionSequence) produce the complete implementation (code + tests).

Which do you want me to produce right now?’

A5. FULL AI RESPONSE AND GENERATED CODE BELOW

All method signatures generated by ChatGPT that I used as template

```
//Main controller / orchestrator (class GameController)

public class GameController {

    // Subprocess 1

    public void initAndRegister() throws InterruptedException;

    // FR-01, FR-02, FR-03

    // Subprocess 2

    public void generateAndRollDice();

    // FR-04

    // Subprocess 3
```

```

public UserTurnResult userTurn() throws InterruptedException;

// FR-05, FR-06, AF-01, AF-02, NFR-03, NFR-04


// Subprocess 4

public void swiftBotTurn() throws MotionException, IOException;

// FR-07, FR-08, AF-03, NFR-07


// Subprocess 5

public Optional<RoundOutcome> checkWinDrawAndAct() throws MotionException,
IOException;

// FR-10 (+trace & LED)


// Subprocess 6

public void appendRoundLogAndUpdateScoreboard(RoundOutcome outcome) throws
IOException;

// FR-11, FR-12, NFR-05


// Subprocess 7

public boolean playAgainOrQuit() throws IOException;

// FR-13 (returns true => play again)

```

//Helper / shared manager classes & signatures

Place these in appropriate classes (Board, MotionService, LoggerService, AIEngine, TimerService, ConfigService).

Board & Move management

```

//Board & Move management

public class Board {

    public void reset();

    public boolean applyMove(Move m); // returns true if applied

    public Move undoLastMove(); // returns undone move or null

    public boolean isCellEmpty(int r, int c);

    public Optional<Piece> checkWinner(List<int[]> outWinningCoords);

```

```

    public boolean isFull();

    public List<Move> snapshotMoveHistory();

    public String asciiBoard(); // UI representation
}

//Motion / SwiftBot interface
public interface SwiftBotInterface {

    void moveTo(int row, int col) throws MotionException, IOException;

    void blink(Color color, int times) throws MotionException, IOException;

    void traceLine(List<int[]> coords) throws MotionException, IOException;

    void spinOnce() throws MotionException, IOException;

    void returnToStart() throws MotionException, IOException;

    boolean isCalibrated(); // calibration check

    CalibrationReport runCalibration() throws IOException;

    //Provide MockSwiftBot implementing the interface for demo (NFR-10).
}

//Logger / Persistence
public class LoggerService {

    public void appendRoundEntry(RoundEntry e);

    public void appendEvent(String csvLine);

    public void flushToDisk(Path filepath) throws IOException; //flush to disk (NFR-05)

    public Path getLogFilePath(); // returns last file path
}

//AI Engine
public class AIEngine {

    public Move chooseRandomMove(Board board, Piece piece);

    public Move chooseStrategicMove(Board board, Piece piece); // minimax/heuristic
}

//Timer service (non-blocking wrapper)
public class TimerService {

    // starts timer and returns a cancel handle; callback is optional

    public CancelToken startTurnTimer(int seconds, Runnable onTimeout);
}

```

```

    public void cancel(CancelToken token);
}

//Config (session)
public class SessionConfig {
    public Difficulty difficulty;
    public int undoQuota;
    public int turnTimerSeconds;
    public boolean enableHardware;
    public boolean aiStrategic;

    // other config items: log dir, calibration tolerances, timeouts
}

//Data types
public class Move { int row; int col; Piece piece; Instant timestamp; }
public enum Piece { X, O }
public enum Color { RED, GREEN, BLUE, YELLOW }
public class RoundOutcome { Optional<Piece> winner; boolean draw; List<Move> moves; }
public class UserTurnResult { enum Type { APPLIED, UNDO, FORFEIT, INVALID, ERROR } ... }
public class MotionException extends Exception { ... }
public class CalibrationReport { boolean ok; double measuredSecondsPer25cm; String notes;
}

//Methods used in multiple flowcharts (reuse & constraints)

// Shared input validation (calls in userTurn and registration)
public ValidationResult validateMoveInput(String input); // parses "r,c" => ints or error reason

// Shared apply move
public boolean applyMoveAndUpdateUI(Player player, Move move) throws IOException;
// called by userTurn() and swiftBotTurn()

// Shared motion command sequence
public void executeMotionSequence(int targetRow, int targetCol, MotionMeta meta) throws
MotionException, IOException;

// used by swiftBotTurn(), checkWinDrawAndAct() (trace), undo reposition

```

```

// Shared scoreboard update (used by appendRoundLogAndUpdateScoreboard and undo)

public void adjustScoreForUndoIfNeeded(Move undoneMove, RoundOutcome
previousOutcome);

// Shared logging helper

public void logMove(Move m); // append to per-round in-memory list

public void logEvent(String event); // append generic event
}

```

The full swiftbot implementation prototype generated by ChatGPT (based on second prompt)

```

public class full_implementation_generated_by_ChatGPT {

    // Save as: NoughtsAndCrosses.java

    import java.io.*;
    import java.nio.file.*;
    import java.time.*;
    import java.time.format.DateTimeFormatter;
    import java.util.*;
    import java.util.concurrent.*;

    /**
     * Noughts & Crosses - Full implementation (Layer 2 logic).
     * - Implements the 7 subprocesses and other methods from Layer 1.
     * - Matches the improved SRS: FRs, NFRs, AFs.
     *
     * Compile: javac NoughtsAndCrosses.java
     * Run:    java NoughtsAndCrosses [--no-hw] [--ai=strategic] [--difficulty=easy|medium|hard]
     */

    public class NoughtsAndCrosses {

        public static void main(String[] args) {

            Config cfg = new Config();

            for (String a : args) {

                if (a.equalsIgnoreCase("--no-hw")) cfg.hardwareEnabled = false;

                if (a.equalsIgnoreCase("--ai=strategic")) cfg.strategicAI = true;
            }
        }
    }
}

```

```

    if (a.startsWith("--difficulty=")) {

        String v = a.substring("--difficulty=".length()).toLowerCase();

        switch (v) {

            case "easy" -> cfg.difficulty = Difficulty.EASY;

            case "hard" -> cfg.difficulty = Difficulty.HARD;

            default -> cfg.difficulty = Difficulty.MEDIUM;

        }

    }

}

GameController controller = new GameController(cfg);

controller.run();

}

/* ----- Shared types & config ----- */

enum Piece { X, O, EMPTY; }

enum Difficulty { EASY, MEDIUM, HARD; }

static class Config {

    public Difficulty difficulty = Difficulty.MEDIUM;

    public int undoQuota = 1;

    public int turnTimerSeconds = 10;

    public boolean hardwareEnabled = false;

    public boolean strategicAI = false;

    public double secondsPer25cm = 1.0; // calibration parameter (default)

}

static class Player {

    public String name;

    public Piece piece;

    public Player(String name, Piece piece) { this.name = name; this.piece = piece; }

}

static class Move {

    public final int row, col;

    public final Piece piece;

```

```

    public final Instant ts;

    public Move(int r, int c, Piece p) { row = r; col = c; piece = p; ts = Instant.now(); }

    public String toCSV() { return String.format("%s|%s|%d|%d", ts.toString(), piece, row, col); }

    public String toString() { return "[" + row + "," + col + "]"; }
}

static class RoundOutcome {

    public Optional<Piece> winner; // EMPTY means draw encoded as Optional.of(EMPTY)

    public List<Move> moves;

    public RoundOutcome(Optional<Piece> winner, List<Move> moves) { this.winner =
winner; this.moves = moves; }

}

/* ----- Board implementation ----- */

static class Board {

    private final Piece[][] grid = new Piece[3][3];

    private final Deque<Move> history = new ArrayDeque<>();

    public Board() { reset(); }

    public synchronized void reset() {

        for (int r=0; r<3; r++) for (int c=0; c<3; c++) grid[r][c] = Piece.EMPTY;

        history.clear();

    }

    public synchronized boolean isEmpty(int r, int c) {

        return grid[r-1][c-1] == Piece.EMPTY;

    }

    public synchronized boolean applyMove(Move m) {

        if (!isEmpty(m.row, m.col)) return false;

        grid[m.row-1][m.col-1] = m.piece;

        history.push(m);

        return true;

    }

    public synchronized Move undoLast() {

```

```

        if (history.isEmpty()) return null;

        Move m = history.pop();

        grid[m.row-1][m.col-1] = Piece.EMPTY;

        return m;
    }

    public synchronized List<Move> getHistory() {
        return new ArrayList<>(history);
    }

    public synchronized Piece at(int r, int c) { return grid[r-1][c-1]; }

    /** checks winner and fills winning coords */
    public synchronized Optional<Piece> checkWinner(List<int[]> winningCoordsOut) {
        // rows
        for (int r=0;r<3;r++) {
            if (grid[r][0] != Piece.EMPTY && grid[r][0]==grid[r][1] && grid[r][1]==grid[r][2]) {
                winningCoordsOut.add(new int[]{r+1,1}); winningCoordsOut.add(new int[]{r+1,2});
                winningCoordsOut.add(new int[]{r+1,3});
                return Optional.of(grid[r][0]);
            }
        }

        // cols
        for (int c=0;c<3;c++) {
            if (grid[0][c] != Piece.EMPTY && grid[0][c]==grid[1][c] && grid[1][c]==grid[2][c]) {
                winningCoordsOut.add(new int[]{1,c+1}); winningCoordsOut.add(new int[]{2,c+1});
                winningCoordsOut.add(new int[]{3,c+1});
                return Optional.of(grid[0][c]);
            }
        }

        // diag
        if (grid[0][0]!=Piece.EMPTY && grid[0][0]==grid[1][1] && grid[1][1]==grid[2][2]) {
            winningCoordsOut.add(new int[]{1,1}); winningCoordsOut.add(new int[]{2,2});
            winningCoordsOut.add(new int[]{3,3});
            return Optional.of(grid[0][0]);
        }
    }

```

```

    }

    if (grid[0][2]!=Piece.EMPTY && grid[0][2]==grid[1][1] && grid[1][1]==grid[2][0]) {
        winningCoordsOut.add(new int[]{1,3}); winningCoordsOut.add(new int[]{2,2});
        winningCoordsOut.add(new int[]{3,1});

        return Optional.of(grid[0][2]);
    }

    return Optional.empty();
}

public synchronized boolean isFull() {
    for (int r=1;r<=3;r++) for (int c=1;c<=3;c++) if (at(r,c) == Piece.EMPTY) return false;
    return true;
}

public synchronized String toAscii() {
    StringBuilder sb = new StringBuilder();
    sb.append("  Columns ->  1  2  3\nRows\n");
    for (int r=1;r<=3;r++) {
        sb.append(" ").append(r).append("    ");
        for (int c=1;c<=3;c++) {
            Piece p = at(r,c);
            sb.append("[").append(p==Piece.EMPTY?" ":p.toString()).append("] ");
        }
        sb.append("\n");
    }
    return sb.toString();
}
}

/* ----- SwiftBot interface & mock ----- */
interface SwiftBotInterface {
    void moveTo(int row, int col) throws IOException;
    void blink(String color, int times) throws IOException;
    void traceLine(List<int[]> coords) throws IOException;
}

```

```

    void spinOnce() throws IOException;

    void returnToStart() throws IOException;

    boolean isCalibrated();

    // Optional: shutdown, emergencyStop
}

static class MockSwiftBot implements SwiftBotInterface {

    public void moveTo(int row, int col) { System.out.println("[Motion] Mock moving to ["+row+";"+col+"]"); }

    public void blink(String color, int times) { System.out.println("[Motion] Mock blink "+color+" x"+times); }

    public void traceLine(List<int[]> coords) { System.out.println("[Motion] Mock trace coords: "+coordsToString(coords)); }

    public void spinOnce() { System.out.println("[Motion] Mock spin once"); }

    public void returnToStart() { System.out.println("[Motion] Mock return to start"); }

    public boolean isCalibrated() { return true; }

    private static String coordsToString(List<int[]> coords) {

        StringBuilder sb = new StringBuilder();

        sb.append("[");

        for (int[] p : coords) sb.append("(" + p[0] + ";" + p[1] + ")");

        sb.append("]");

        return sb.toString();

    }

}

/* ----- LoggerService ----- */

static class LoggerService {

    private final List<String> entries = Collections.synchronizedList(new ArrayList<>());

    private final Path dir;

    public LoggerService() {

        String home = System.getProperty("user.home");

        dir = Paths.get(home, "Documents", "swiftbot_logs");
    }
}

```

```

        try { Files.createDirectories(dir); } catch (IOException ignored) {}
    }

    public void append(String csv) { entries.add(csv); }

    public synchronized void flushToDisk(String filename) throws IOException {
        if (entries.isEmpty()) return;

        Path path = dir.resolve(filename);

        try (BufferedWriter w = Files.newBufferedWriter(path, StandardOpenOption.CREATE,
StandardOpenOption.APPEND)) {

            for (String s : entries) w.write(s + System.lineSeparator());

            w.flush();

            entries.clear();

        }
    }

    public String samplePath(String filename) { return
dir.resolve(filename).toAbsolutePath().toString(); }

}

/* ----- AI engine (random + minimax) ----- */
static class ALEngine {

    private final Random rnd = new Random();

    private final boolean strategic;

    public ALEngine(boolean strat) { this.strategic = strat; }

    public Move chooseMove(Board board, Piece piece) {

        if (!strategic) return chooseRandom(board, piece);

        return chooseMinimax(board, piece);

    }

    private Move chooseRandom(Board board, Piece piece) {

        List<int[]> empties = new ArrayList<>();

        for (int r=1; r<=3; r++) for (int c=1; c<=3; c++) if (board.at(r,c)==Piece.EMPTY)
empties.add(new int[]{r,c});

        if (empties.isEmpty()) return null;

        int[] rc = empties.get(rnd.nextInt(empties.size()));

```

```

        return new Move(rc[0], rc[1], piece);
    }

    private Move chooseMinimax(Board board, Piece piece){
        Piece opponent = (piece == Piece.X ? Piece.O : Piece.X);
        int bestScore = Integer.MIN_VALUE;
        Move best = null;
        for (int r=1;r<=3;r++) for (int c=1;c<=3;c++) {
            if (board.at(r,c)!=Piece.EMPTY) continue;
            Move trial = new Move(r,c,piece);
            board.applyMove(trial);
            int score = minimax(board, 0, false, piece, opponent);
            board.undoLast();
            if (score > bestScore) { bestScore = score; best = trial; }
        }
        return best != null ? best : chooseRandom(board, piece);
    }

    private int minimax(Board board, int depth, boolean isMax, Piece aiPiece, Piece
humanPiece) {
        List<int[]> dump = new ArrayList<>();
        Optional<Piece> winner = board.checkWinner(dump);
        if (winner.isPresent()) {
            if (winner.get() == aiPiece) return 10 - depth;
            if (winner.get() == humanPiece) return depth - 10;
        }
        if (board.isFull()) return 0;
        int best = isMax ? Integer.MIN_VALUE : Integer.MAX_VALUE;
        Piece turn = isMax ? aiPiece : humanPiece;
        for (int r=1;r<=3;r++) for (int c=1;c<=3;c++) {
            if (board.at(r,c) != Piece.EMPTY) continue;
            Move mv = new Move(r,c,turn);

```

```

        board.applyMove(mv);

        int val = minimax(board, depth+1, !isMax, aiPiece, humanPiece);

        board.undoLast();

        if (isMax) best = Math.max(best, val); else best = Math.min(best, val);
    }

    return best;
}
}

/* ----- GameController: implements Layer 2 logic (subprocesses) ----- */
static class GameController {

    private final Config cfg;

    private final Board board = new Board();

    private final LoggerService logger = new LoggerService();

    private SwiftBotInterface swiftBot;

    private AIEngine aiEngine;

    private final Scanner in = new Scanner(System.in);

    private Player user, bot, currentPlayer;

    private int userUndoRemaining;

    private int roundNumber = 0;

    private int scoreUser = 0;

    private int scoreBot = 0;

    public GameController(Config cfg) {

        this.cfg = cfg;

        this.swiftBot = cfg.hardwareEnabled ? new MockSwiftBot() : new MockSwiftBot(); //
        placeholder for real driver

        this.aiEngine = new AIEngine(cfg.strategicAI);

        applyDifficulty(cfg.difficulty);
    }

    private void applyDifficulty(Difficulty d) {

```

```

    cfg.difficulty = d;

    switch (d) {

        case EASY -> { cfg.turnTimerSeconds = 0; cfg.undoQuota = 2; }

        case MEDIUM -> { cfg.turnTimerSeconds = 10; cfg.undoQuota = 1; }

        case HARD -> { cfg.turnTimerSeconds = 5; cfg.undoQuota = 0; }

    }

}

/* SUBPROCESS 1: init & register */

public void initAndRegister() {

    System.out.println();

    System.out.println("Microsoft Windows [Version 10.0]");

    System.out.println("(c) 2026 SwiftBot Labs. All rights reserved.");

    System.out.println("=====");

    System.out.println(" NOUGHTS & CROSSES — SwiftBot (CLI)");

    System.out.println("=====\\n");

    System.out.println("Welcome! Play Noughts & Crosses (3x3) vs SwiftBot.");

    System.out.println("Rules (brief):");

    System.out.println(" - Enter moves as row,column using numbers 1..3 (example: 2,3)");

    System.out.println(" - Difficulty sets Undo quota & timer");

    System.out.println(" - Use 'undo' to revert your last move (if available)");

    System.out.print("\\nPress SwiftBot Button A (or type anything + Enter) to begin... ");

    in.nextLine(); // accept any input for demo start

    // registration

    String name;

    while (true) {

        System.out.print("\\nEnter your player name (alphanumeric, 1–30 chars):\\n> ");

        name = in.nextLine().trim();

        if (name.matches("[A-Za-z0-9 ]{1,30}")) break;

        System.out.println("[ERROR] Invalid name. Use letters, digits and spaces (1–30).");

    }
}

```

```

user = new Player(name, Piece.X);

System.out.println("[OK] Player name set: " + name);

String botName = "Robo-" + (100 + new Random().nextInt(900));
System.out.println("SwiftBot default name: \"\" + botName + "\"");
System.out.print("Override SwiftBot name? (Y/N) > ");
String yn = in.nextLine().trim();
if (yn.equalsIgnoreCase("Y")) {
    System.out.print("Enter SwiftBot name > ");
    botName = in.nextLine().trim();
}
bot = new Player(botName, Piece.O);
System.out.println("[OK] SwiftBot name: " + botName);
}

/* SUBPROCESS 2: generate & roll dice */
public void generateAndRollDice() {
    Random r = new Random();
    int ur, br;
    do {
        ur = r.nextInt(6) + 1;
        br = r.nextInt(6) + 1;
        System.out.printf("[%s] rolled: %d\n", user.name, ur);
        System.out.printf("[%s] rolled: %d\n", bot.name, br);
        if (ur == br) System.out.println("Tie — re-rolling...");
    } while (ur == br);
    if (ur > br) {
        user.piece = Piece.O; bot.piece = Piece.X; currentPlayer = user;
    } else {
        bot.piece = Piece.O; user.piece = Piece.X; currentPlayer = bot;
    }
    System.out.printf("Starter: %s (%s)\n", currentPlayer==user ? user.name : bot.name,

```

```

        currentPlayer==user ? user.piece : bot.piece);
    }

    /* SUBPROCESS 3: user's turn */

    public boolean userTurn() {

        // Display board and prompt

        System.out.println(board.toAscii());

        System.out.printf("Current turn: %s (%s)\n", currentPlayer==user?user.name:bot.name,
currentPlayer.piece);

        System.out.printf("Undo moves left: %d   Time limit: %d seconds\n",
userUndoRemaining, cfg.turnTimerSeconds);

        System.out.print("Your move (format row,column) or 'undo' > ");

        ExecutorService ex = Executors.newSingleThreadExecutor();

        Future<String> future = ex.submit(() -> {

            try { return in.nextLine().trim(); } catch (Exception e) { return ""; }

        });

        String input = null;

        try {

            if (cfg.turnTimerSeconds > 0) input = future.get(cfg.turnTimerSeconds,
TimeUnit.SECONDS);

            else input = future.get();

        } catch (TimeoutException te) {

            future.cancel(true);

            System.out.println("*** TIME EXPIRED — turn forfeited ***");

            logger.append("FORFEIT,"+user.name+"-"+Instant.now().toString());

            ex.shutdownNow();

            return false; // forfeit => AI's turn

        } catch (Exception e) {

            future.cancel(true);

            System.out.println("[ERROR] Input error. Try again.");

            ex.shutdownNow();

            return false;

        } finally {

```

```

        ex.shutdownNow();
    }

    if (input.equalsIgnoreCase("undo")) {
        return handleUndo(); // true if undo performed (counts as action)
    }

    // parse move
    String[] parts = input.split(" ");

    if (parts.length != 2) {
        System.out.println("[ERROR] Invalid format. Use row,column (e.g., 2,3).");
        return userTurn();
    }

    int r,c;

    try { r = Integer.parseInt(parts[0].trim()); c = Integer.parseInt(parts[1].trim()); }
    catch (NumberFormatException nfe) { System.out.println("[ERROR] Non-numeric input. Try again."); return userTurn(); }

    if (r<1 || r>3 || c<1 || c>3) { System.out.println("[ERROR] Row/Col must be 1..3."); return userTurn(); }

    if (!board.isEmpty(r,c)) { System.out.printf("[ERROR] Square [%d,%d] occupied. Try again.\n", r, c); return userTurn(); }

    Move m = new Move(r,c,user.piece);

    boolean ok = applyMoveAndUpdateUI(user, m);

    return ok;
}

/* SUBPROCESS 4: swiftbot's turn */
public void swiftBotTurn() {
    Move m = aiEngine.chooseMove(board, bot.piece);

    if (m == null) { System.out.println("[INFO] No valid moves for SwiftBot."); return; }

    System.out.printf("[%s - %s] will move to square %s\n", bot.name, bot.piece, m);

    try {
        executeMotionSequence(m.row, m.col, "GREEN", 3);

        boolean ok = applyMoveAndUpdateUI(bot, m);

        if (!ok) {

```

```

        System.out.println("[ERROR] Failed to apply SwiftBot move to internal board.");
    }
} catch (IOException ex) {
    System.out.println("[ERROR] Motion error: " + ex.getMessage());
    logger.append("MOTION_FAILURE,"+ex.getMessage()+","+Instant.now().toString());
}
}

/* SUBPROCESS 5: check win/draw and act (trace, blink) */
public Optional<Piece> checkWinDrawAndAct() {
    List<int[]> winCoords = new ArrayList<>();

    Optional<Piece> w = board.checkWinner(winCoords);

    if (w.isPresent()) {
        Piece winner = w.get();

        if (winner == user.piece) {
            System.out.println("*** WINNER: " + user.name + " (" + winner + ") ***");

            try {
                executeMotionSequenceForTrace(winCoords, "GREEN",3);
            } catch (IOException e) { System.out.println("[ERROR] Motion during win trace: " +
e.getMessage()); }

            scoreUser++;
        } else {
            System.out.println("*** WINNER: " + bot.name + " (" + winner + ") ***");

            try { executeMotionSequenceForTrace(winCoords, "RED",3); } catch (IOException e) {
System.out.println("[ERROR] Motion during win trace: " + e.getMessage()); }

            scoreBot++;
        }
    }

    return Optional.of(winner);
} else if (board.isFull()) {
    System.out.println("*** ROUND RESULT: DRAW ***");

    try { swiftBot.spinOnce(); swiftBot.blink("BLUE", 3); } catch (IOException e) { /* log */ }

    return Optional.of(Piece.EMPTY);
} else {

```

```

        return Optional.empty();
    }
}

/* SUBPROCESS 6: append log and update scoreboard */
public void appendRoundLogAndUpdateScoreboard(Optional<Piece> outcome) {
    roundNumber++;

    String result = "DRAW";

    String winnerName = "";

    if (outcome.isPresent() && outcome.get() != Piece.EMPTY) {
        result = outcome.get() == user.piece ? "USER_WIN" : "SWIFTBOT_WIN";

        winnerName = outcome.get() == user.piece ? user.name : bot.name;
    }

    // build move sequence CSV-ish
    List<Move> hist = board.getHistory();

    StringBuilder seq = new StringBuilder();

    for (int i = hist.size()-1; i>=0; i--) { Move m = hist.get(i);
seq.append(m.toCSV()).append(";"); }

    String csvLine = String.join(";", Integer.toString(roundNumber), seq.toString(), result,
winnerName, Instant.now().toString());

    logger.append(csvLine);

    // display scoreboard

    System.out.println("--- Round summary ---");

    System.out.printf("Round #: %d\nResult: %s\n", roundNumber, result);

    System.out.printf("Scoreboard:\n %s: %d\n %s: %d\n", user.name, scoreUser,
bot.name, scoreBot);
}

/* SUBPROCESS 7: play again or quit */
public boolean playAgainOrQuit() {

    System.out.print("Play again? (Y=play again / X=quit) > ");

    String s = in.nextLine().trim();

    if (s.equalsIgnoreCase("Y")) {

        board.reset();

```

```

        userUndoRemaining = cfg.undoQuota;

        return true;
    } else {

        String fname = "noughts_log_" +
DateTimeFormatter.ofPattern("yyyyMMdd_HHmmss").format(LocalDateTime.now()) + ".csv";

        try { logger.flushToDisk(fname); System.out.println("[OK] Log saved to: " +
logger.samplePath(fname)); }

        catch (IOException e) { System.out.println("[ERROR] Failed to write log: " +
e.getMessage()); }

        System.out.println("Program terminating. Goodbye.");

        return false;
    }
}

/* Helper: apply move & update UI atomically */
private synchronized boolean applyMoveAndUpdateUI(Player p, Move m) {
    boolean applied = board.applyMove(m);

    if (!applied) return false;

    // Log move in memory (we keep per-round sequence via board.history)

    // Print summary and ASCII board

    System.out.printf("[%s - %s] moved to square %s\n", p.name, p.piece, m);

    System.out.println(board.toAscii());

    return true;
}

/* Undo handler (AF-01) - user-only undo */
private synchronized boolean handleUndo() {
    if (userUndoRemaining <= 0) {

        System.out.println("Undo moves remaining: 0. Cannot undo.");

        return false;
    }

    System.out.print("Confirm undo last move? (Y/N) > ");

    String conf = in.nextLine().trim();

    if (!conf.equalsIgnoreCase("Y")) return false;

```

```

// Undo only user's last move (per SRS: last move the player makes)
// We must undo moves until last move belonging to user is removed
Move undone = null;
List<Move> temp = board.getHistory();
if (temp.isEmpty()) { System.out.println("Nothing to undo."); return false; }
// If latest move belongs to bot, we should not undo it (spec says player's last move)
// We'll check history; if last move is bot's, we undo twice: bot then user? To obey SRS:
only user's last move
// Implementation decision: undo last move only if it was by the user; otherwise inform
user
Move last = temp.get(temp.size()-1);
if (last.piece != user.piece) {
    System.out.println("Last move was not yours. Cannot undo until after your move.");
    return false;
} else {
    undone = board.undoLast();
}
if (undone != null) {
    userUndoRemaining--;
    logger.append("UNDO," + undone.toString() + "," + Instant.now().toString());
    System.out.println("[OK] Last move undone. Undo left: " + userUndoRemaining);
    System.out.println(board.toAscii());
    return true;
} else {
    System.out.println("Nothing to undo.");
    return false;
}
}

/* Execute motion sequence for a move (movement + blink + return) */
private void executeMotionSequence(int targetRow, int targetCol, String blinkColor, int
blinkTimes) throws IOException {
    // Calibration check

```

```

        if (!swiftBot.isCalibrated()) throw new IOException("Calibration out of bounds");

        // send motion

        swiftBot.moveTo(targetRow, targetCol);

        swiftBot.blink(blinkColor, blinkTimes);

        swiftBot.returnToStart();

        // Note: actual timing/telemetry not implemented for mock
    }

    /* Execute motion for tracing line */

    private void executeMotionSequenceForTrace(List<int[]> coords, String blinkColor, int
blinkTimes) throws IOException {

        if (!swiftBot.isCalibrated()) throw new IOException("Calibration out of bounds");

        swiftBot.blink(blinkColor, blinkTimes);

        swiftBot.traceLine(coords);

        swiftBot.blink(blinkColor, blinkTimes);

    }

    /* Main run: orchestrates subprocesses */

    public void run() {

        initAndRegister();

        System.out.print("Choose difficulty (easy/medium/hard) > ");

        String d = in.nextLine().trim().toLowerCase();

        if (d.equals("easy")) applyDifficulty(Difficulty.EASY);

        else if (d.equals("hard")) applyDifficulty(Difficulty.HARD);

        else applyDifficulty(Difficulty.MEDIUM);

        System.out.printf("Difficulty: %s (Undo=%d, Timer=%ds)\n", cfg.difficulty,
cfg.undoQuota, cfg.turnTimerSeconds);

        generateAndRollDice();

        boolean keepPlaying = true;

        userUndoRemaining = cfg.undoQuota;

        while (keepPlaying) {

            board.reset();

            boolean roundOver = false;

```

```

while (!roundOver) {
    if (currentPlayer == user) {
        boolean applied = userTurn();

        Optional<Piece> outcome = checkWinDrawAndAct();

        if (outcome.isPresent()) { appendRoundLogAndUpdateScoreboard(outcome);
roundOver = true; break; }

        currentPlayer = bot;
    } else {
        swiftBotTurn();

        Optional<Piece> outcome = checkWinDrawAndAct();

        if (outcome.isPresent()) { appendRoundLogAndUpdateScoreboard(outcome);
roundOver = true; break; }

        currentPlayer = user;
    }
}

keepPlaying = playAgainOrQuit();
}

```

Appendix B (Claude AI)

B1. TOOL USED

Anthropic Claude sonnet 4.6 extended thinking

B2. PURPOSE OF USE

to help me fix issues like how to implement the swiftbot button event handling and to help me refactor and simplify my program by removing unnecessary classes, methods and variables that would clutter my code. By helping me simplifying my program means making it more readable and approachable so it reduced unnecessary complexity and improve modularity.

B3. ALL PROMPTS PROVIDED

- 1) *'Based on the uploaded files, I cannot get my program (nought and crosses) to respond to button clicks and LED lights. based on the improved SRS uploaded and the DoesMyswiftbotwork template for button event handlings and LED lights, can you show me how I can implement these in my program please as the last time I run the program on my computer (along with the swiftbot (based on raspberry pi) API) in ssh, the program only responds to keyboard events but cannot also do the same for button events as well as lighting up the appropriate LEDs because my SRS and game needs. so how can I fix this'*
- 2) *'can you explain how I can make my program much simpler (only keep the 7 subprocesses methods under the gamecontroller class) without using confusing and*

unknown built-system methods, unknown objects and classes, and keeping it as simple and modular as possible for a beginners attempt to build a program like this'

- 3) 'can you show me how this can be implemented in the program fully and left me review in and ensure the program stays simple as possible'

B4. FULL AI-GENERATED RESPONSE

Prompt 1

'Now I have a clear picture of the issues. There are three root problems:

1. **The import swiftbot.*; is missing** on line 8 — it's just import with nothing after it.
2. **GameController always uses MockSwiftBot** even when --no-hw isn't passed — the real SwiftBot driver class is never implemented.
3. **Button events are never registered** — the game only calls in.nextLine(), so the SwiftBot buttons do nothing.'

Here's a breakdown of every change made and why:

1. Fixed the broken import (line 8) The line was just import with nothing after it, which is a compile error. Changed to import swiftbot.*; so all SwiftBot API classes (Button, Underlight, SwiftBotAPI, etc.) are available.

2. Added RealSwiftBotDriver class This is the main new addition. It implements your existing SwiftBotInterface using the real SwiftBot API. The key methods are:

- **waitForButton(Button)** — lights the button's correct LED colour per SRS 2.2 (A=Red, B=Green, X=Blue, Y=Yellow), then uses enableButton() + a CountdownLatch to block until the physical button is pressed. This is exactly the pattern from your DoesMySwiftBotWork.java template.
 - **waitForYorX()** — lights both Y (Yellow) and X (Blue) simultaneously and returns whichever is pressed first, for the end-of-round menu.
 - **blink(), moveTo(), spinOnce(), returnToStart(), traceLine()** are all wired to the real API.
-

3. Fixed GameController constructor Previously it read:

java

```
this.swiftBot = cfg.hardwareEnabled ? new MockSwiftBot() : new MockSwiftBot(); // placeholder
```

Both branches were MockSwiftBot! Now it instantiates RealSwiftBotDriver when hardware is enabled, with a safe fallback to Mock if initialisation fails.

A realBot() helper casts to RealSwiftBotDriver so button methods are accessible without changing the interface.

4. Fixed `initAndRegister()` *The old line just called `in.nextLine()` and printed "Press Button A". Now it actually calls `rb.waitForButton(Button.A)`, which lights the red LED and blocks until the physical button is pressed.*

5. Fixed `playAgainOrQuit()` *Now calls `rb.waitForYorX()`, which lights Button Y (yellow) and Button X (blue) simultaneously and responds to whichever is pressed first per SRS 2.9.*

6. Flipped `hardwareEnabled` default to `true` *Since you're running on the SwiftBot over SSH, hardware should be on by default. Add `--no-hw` as a run argument when you want to test on your laptop without the bot connected.'*

B5. GENERATED FULL CODE BELOW BASED ON THE PROMPTS GIVEN

```
package full_software_implementation;

import java.io.*;
import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;
import java.util.*;
import swiftbot.*;

public class NoughtsAndCrosses {

    // -----
    // ANSI colour codes for console output
    // -----

    static final String RESET = "\u001B[0m";
    static final String RED   = "\u001B[31m";
    static final String GREEN = "\u001B[32m";
    static final String YELLOW = "\u001B[33m";
    static final String ORANGE = "\u001B[38;2;255;165;0m";
    static final String PURPLE = "\u001B[35m";

    // -----
    // Entry point — just creates the game and runs it
    // -----

    public static void main(String[] args) {

        GameController game = new GameController();
```

```

    game.run();
}

// =====

// GAME CONTROLLER

// All game state lives here as plain fields.

// All game logic lives in the 7 methods below.

// =====

static class GameController {

    // --- board: a 3x3 grid of strings, each cell is "X", "O" or " " ---
    String[][] board = new String[3][3];

    // --- player info ---
    String userName = "";
    String botName = "";
    String userPiece = ""; // "X" or "O" — assigned after dice roll
    String botPiece = ""; // "X" or "O"
    String currentTurn = ""; // "user" or "bot"

    // --- scores ---
    int scoreUser = 0;
    int scoreBot = 0;
    int roundNumber = 0;

    // --- difficulty settings ---
    String difficulty = "medium";
    int undoQuota = 1; // how many undos the user gets per round
    int timerSeconds = 10; // 0 means no timer (easy mode)
    int undoRemaining = 0; // countdown during a round

    // --- undo tracking: stores the row/col of the user's last move ---

```

```

int lastUserRow = -1; // -1 means no move has been made yet

int lastUserCol = -1;

// --- log: a simple list of strings written to file at the end ---
List<String> logEntries = new ArrayList<>();

// --- SwiftBot hardware ---
SwiftBotAPI api;
boolean hardwareOn = false;

// --- shared Scanner (one instance for the whole program) ---
Scanner in = new Scanner(System.in);

// --- random number generator for dice rolls and bot moves ---
Random rnd = new Random();

// -----
// Constructor — tries to connect to the SwiftBot hardware.
// If it fails, the game still runs in keyboard-only mode.
// -----
public GameController() {
    try {
        api = SwiftBotAPI.INSTANCE;
        hardwareOn = true;
        System.out.println("[HW] SwiftBot connected successfully.");
    } catch (Exception e) {
        System.out.println(ORANGE + "[WARNING] SwiftBot not available. Running in keyboard-
only mode." + RESET);
        hardwareOn = false;
    }
}

```

```

}

// =====

// METHOD 1 — initAndRegister

// Shows the welcome screen, waits for Button A, and registers
// the player name and SwiftBot name.

// SRS ref: 2.3

// =====

void initAndRegister() {

    System.out.println("=====");
    System.out.println("----- NOUGHTS & CROSSES -----");
    System.out.println("=====\\n");
    System.out.println("Welcome! Play Noughts & Crosses (3x3) against SwiftBot.");
    System.out.println();
    System.out.println("-----");
    System.out.println("          GAME RULES:");
    System.out.println("-----");
    System.out.println(" 1) Enter moves as row,column - e.g. 2,3 - integers only.");
    System.out.println(" 2) Three difficulty modes: easy / medium / hard.");
    System.out.println(" 3) Timer only applies to medium (10s) and hard (5s).");
    System.out.println(" 4) Type 'undo' to revert your last move (if available).");
    System.out.println(" 5) Undos per round: easy = 2 | medium = 1 | hard = 0.");
    System.out.println(" 6) Get 3 in a row (horizontal, vertical, diagonal) to win.");
    System.out.println(" 7) A draw = SwiftBot spins once and blinks blue 3 times.");
    System.out.println("-----\\n");

    // --- wait for Button A (SRS 2.3b) ---

    // Button A LED is Red (SRS 2.2)

    System.out.print("[INPUT] Press Button A on the SwiftBot to begin (or press Enter): ");

    if (hardwareOn) {

```

```

try {
    // light Button A red, then wait until it is physically pressed
    api.setButtonLight(Button.A, true);
    boolean[] pressed = { false };
    api.enableButton(Button.A, () -> {
        pressed[0] = true;
        api.disableButton(Button.A);
        api.setButtonLight(Button.A, false);
    });
    while (!pressed[0]) {
        Thread.sleep(100); // check every 100ms
    }
    System.out.println("\n[PROCESS] Button A pressed - starting registration...");
} catch (Exception e) {
    in.nextLine(); // hardware error: fall back to keyboard
}

} else {
    in.nextLine(); // keyboard-only fallback
}

// --- get username (SRS 2.3c) ---
while (true) {
    System.out.println();
    System.out.print("[INPUT] Enter your username (lowercase letters/numbers, 1-15
chars):\n> ");
    String name = in.nextLine().trim();
    // only lowercase letters and digits, 1 to 15 characters
    if (name.matches("[a-z0-9]{1,15}$")) {
        userName = name;
        break;
    }
}

```

```

        System.out.println(RED + "*** [ERROR] Invalid username. Lowercase letters and
numbers only, max 15 chars. ***" + RESET);
    }

    System.out.println(YELLOW + "[SUCCESS] Username set: " + RESET + userName);

    // --- generate or override bot name (SRS 2.3d) ---
    String generatedName = "Robo-" + (100 + rnd.nextInt(900));
    System.out.println("\n[INFO] SwiftBot generated name: \"" + generatedName + "\"");
    System.out.print("[INPUT] Override SwiftBot name? (yes/no):\n> ");
    String yn = in.nextLine().trim();
    if (yn.equalsIgnoreCase("yes")) {
        System.out.print("[INPUT] Enter SwiftBot name:\n> ");
        botName = in.nextLine().trim();
    } else {
        botName = generatedName;
    }
    System.out.println(YELLOW + "[SUCCESS] SwiftBot name set: " + RESET + botName);
}

// =====

// METHOD 2 — rollDice

// Rolls a virtual 6-sided die for both players. The higher roll
// goes first and gets the "O" piece. Re-rolls on a tie.
// SRS ref: 2.4
// =====

void rollDice() {
    System.out.println("\n[PROCESS] Rolling dice to decide who goes first...\n");

    int userRoll, botRoll;

    do {

```

```

        userRoll = rnd.nextInt(6) + 1; // random number 1-6

        botRoll = rnd.nextInt(6) + 1;

        System.out.println "[" + userName + "] rolled: " + userRoll);
        System.out.println "[" + botName + "] rolled: " + botRoll);

        if (userRoll == botRoll) {
            System.out.println("Tie - re-rolling...\n");
        }
    } while (userRoll == botRoll);

    // highest roll gets "O" and goes first (SRS 2.4b)
    if (userRoll > botRoll) {
        userPiece = "O";
        botPiece = "X";
        currentTurn = "user";
    } else {
        botPiece = "O";
        userPiece = "X";
        currentTurn = "bot";
    }

    String starterName = currentTurn.equals("user") ? userName : botName;
    String starterPiece = currentTurn.equals("user") ? userPiece : botPiece;

    System.out.println(PURPLE + "\n[RESULT] " + starterName + " goes first with piece: " +
        starterPiece + RESET);
}

// =====

// METHOD 3 — userTurn

// Prompts the user for a move, validates it, and places the piece.

// Uses a while loop (not recursion) so invalid inputs just re-prompt.

```

```

// Handles the timer countdown and undo requests too.

// Returns true if a piece was placed, false if the turn was forfeited.

// SRS ref: 2.5

// =====

boolean userTurn() {

    // calculate the deadline for this turn (Long.MAX_VALUE = no limit)
    long deadline = (timerSeconds > 0)
        ? System.currentTimeMillis() + (timerSeconds * 1000L)
        : Long.MAX_VALUE;

    while (true) {

        printBoard();

        System.out.println("-----");

        System.out.println("[INFO] Your turn: " + userName + " (" + userPiece + ")");

        System.out.println("[INFO] Undos left: " + undoRemaining + " Time limit: " +
timerSeconds + "s");

        System.out.print("[INPUT] Enter move (row,column) or 'undo':\n> ");

        // --- wait for the user to type something, checking the timer ---
        try {
            while (System.in.available() == 0) {
                if (System.currentTimeMillis() >= deadline) {
                    System.out.println(ORANGE + "\n--- [WARNING] Time ran out! Your turn is
forfeited. ---" + RESET);

                    logEntries.add("FORFEIT," + userName + "," + LocalDateTime.now());

                    return false; // forfeit - hand turn to the bot
                }

                Thread.sleep(100); // wait 100ms then check again
            }
        } catch (Exception e) {

```

```

        return false;
    }

    String input = in.nextLine().trim();

    // ignore blank lines
    if (input.isEmpty()) {
        continue;
    }

    // --- handle undo request ---
    if (input.equalsIgnoreCase("undo")) {
        handleUndo();
        continue; // re-prompt for a move on the same turn
    }

    // --- parse the row,column input ---
    String[] parts = input.split(",");
    if (parts.length != 2) {
        System.out.println(RED + "*** [ERROR] Invalid format. Use row,column e.g. 2,3 ***" +
RESET);
        continue;
    }

    int r, c;
    try {
        r = Integer.parseInt(parts[0].trim());
        c = Integer.parseInt(parts[1].trim());
    } catch (NumberFormatException e) {
        System.out.println(RED + "*** [ERROR] Numbers only please. ***" + RESET);
        continue;
    }

```

```

    }

    if (r < 1 || r > 3 || c < 1 || c > 3) {
        System.out.println(RED + "*** [ERROR] Row and column must each be between 1 and
3. ***" + RESET);
        continue;
    }

    if (!board[r - 1][c - 1].equals(" ")) {
        System.out.println(RED + "*** [ERROR] Square [" + r + ", " + c + "] is already taken. Try
again. ***" + RESET);
        continue;
    }

    // --- valid move: place the piece ---
    board[r - 1][c - 1] = userPiece;

    lastUserRow = r; // remember position in case user wants to undo
    lastUserCol = c;

    System.out.println(PURPLE + "[RESULT] " + userName + " (" + userPiece + ") moved to ["
+ r + ", " + c + "]" + RESET);

    logEntries.add(userName + ", " + userPiece + ", " + r + ", " + c + ", " + LocalDateTime.now());

    return true;
}
}

// -----
// handleUndo — helper called from userTurn only
// Removes the user's last placed piece from the board.
// SRS ref: 4.1
// -----

void handleUndo() {
    if (undoRemaining <= 0) {

```

```

        System.out.println(ORANGE + "--- [WARNING] No undos remaining. ---" + RESET);
        return;
    }
    if (lastUserRow == -1) {
        System.out.println(ORANGE + "--- [WARNING] No move to undo yet. ---" + RESET);
        return;
    }
    System.out.print("[INPUT] Confirm undo? (yes/no):\n> ");
    String conf = in.nextLine().trim();
    if (!conf.equalsIgnoreCase("yes")) {
        return;
    }

    // clear the last user move from the board
    board[lastUserRow - 1][lastUserCol - 1] = " ";
    undoRemaining--;
    lastUserRow = -1;
    lastUserCol = -1;

    System.out.println(YELLOW + "[SUCCESS] Move undone. Undos left: " + undoRemaining +
RESET);
    printBoard();
}

// =====

// METHOD 4 — swiftBotTurn
// Picks a random empty square, moves the SwiftBot there,
// blinks green 3 times, then returns to start.
// SRS ref: 2.6
// =====

void swiftBotTurn() {

```

```

// build a list of all empty squares

List<int[]> emptySquares = new ArrayList<>();

for (int r = 1; r <= 3; r++) {
    for (int c = 1; c <= 3; c++) {
        if (board[r - 1][c - 1].equals(" ")) {
            emptySquares.add(new int[]{ r, c });
        }
    }
}

if (emptySquares.isEmpty()) {
    return; // board is full - no move to make
}

// pick a random empty square (SRS 2.6a)
int[] chosen = emptySquares.get(rnd.nextInt(emptySquares.size()));
int r = chosen[0];
int c = chosen[1];

System.out.println("\n[INFO] " + botName + " (" + botPiece + ") will move to [" + r + "," + c +
    "]);

// --- SwiftBot physical movement (SRS 2.6b-d) ---
if (hardwareOn) {
    try {
        int msPerCell = 1000; // adjust to match your board (1 second per 25cm)

        // move forward and sideways to reach the target square
        api.move(100, 100, r * msPerCell);

        Thread.sleep(300);
    }
}

```

```

        api.move(100, -100, c * msPerCell);

        Thread.sleep(300);

        // blink green 3 times on arrival (SRS 2.6c)
        blinkUnderlights("GREEN", 3);

        // return to start position (SRS 2.6d)
        api.move(-100, 100, c * msPerCell);
        Thread.sleep(300);
        api.move(-100, -100, r * msPerCell);
        Thread.sleep(300);

    } catch (Exception e) {
        System.out.println(RED + "**** [ERROR] SwiftBot motion error: " + e.getMessage() + "
***" + RESET);
    }
}

// place the piece on the board
board[r - 1][c - 1] = botPiece;

System.out.println(PURPLE + "[RESULT] " + botName + " (" + botPiece + ") moved to [" + r +
";" + c + "]" + RESET);

logEntries.add(botName + ";" + botPiece + ";" + r + ";" + c + ";" + LocalDateTime.now());
printBoard();
}

// =====
// METHOD 5 — checkWinOrDraw
// Checks every winning line on the board.
// Returns "X" or "O" if someone won, "DRAW" if the board is full,
// or null if the game is still going.

```

```

// SRS ref: 2.7

// =====

String checkWinOrDraw() {

    // check all 3 rows
    for (int r = 0; r < 3; r++) {
        if (!board[r][0].equals(" "))
            && board[r][0].equals(board[r][1])
            && board[r][1].equals(board[r][2])) {
            return board[r][0]; // winning piece
        }
    }

    // check all 3 columns
    for (int c = 0; c < 3; c++) {
        if (!board[0][c].equals(" "))
            && board[0][c].equals(board[1][c])
            && board[1][c].equals(board[2][c])) {
            return board[0][c];
        }
    }

    // check top-left to bottom-right diagonal
    if (!board[0][0].equals(" "))
        && board[0][0].equals(board[1][1])
        && board[1][1].equals(board[2][2])) {
        return board[0][0];
    }

    // check top-right to bottom-left diagonal
    if (!board[0][2].equals(" "))

```

```

        && board[0][2].equals(board[1][1])
        && board[1][1].equals(board[2][0])) {
    return board[0][2];
}

// if no empty squares remain and no winner was found, it is a draw
for (int r = 0; r < 3; r++) {
    for (int c = 0; c < 3; c++) {
        if (board[r][c].equals(" ")) {
            return null; // still empty squares - game continues
        }
    }
}
return "DRAW";
}

// -----
// handleOutcome — helper called from run() after checkWinOrDraw
// Triggers the correct LED blink / spin sequence and updates scores.
// SRS ref: 2.7a-c
// -----

void handleOutcome(String result) {
    if (result.equals("DRAW")) {
        System.out.println("\n***** DRAW! *****");
        // SRS 2.7c: spin once and blink blue 3 times
        if (hardwareOn) {
            spinOnce();
            blinkUnderlights("BLUE", 3);
        }
    } else if (result.equals(userPiece)) {
        System.out.println("\n***** " + userName + " WINS! *****");
        scoreUser++;
    }
}

```

```

        // SRS 2.7a: blink green 3 times for user win
        if (hardwareOn) {
            blinkUnderlights("GREEN", 3);
        }
    } else {
        System.out.println("\n***** " + botName + " WINS! *****");
        scoreBot++;

        // SRS 2.7b: blink red 3 times for SwiftBot win
        if (hardwareOn) {
            blinkUnderlights("RED", 3);
        }
    }
}

// =====

// METHOD 6 — saveLogAndShowScore
// Appends the round result to the log list and prints the scoreboard.
// The full log list is written to disk when the player quits.
// SRS ref: 2.8
// =====

void saveLogAndShowScore(String result) {
    roundNumber++;

    // decide the winner label for the log
    String winnerLabel;
    if (result.equals("DRAW")) {
        winnerLabel = "DRAW";
    } else if (result.equals(userPiece)) {
        winnerLabel = userName;
    } else {

```

```

        winnerLabel = botName;
    }

    // add this round's summary to the log list (written to file on quit)
    String logLine = "Round " + roundNumber + ","
        + winnerLabel + ","
        + result + ","
        + LocalDateTime.now().toString();
    logEntries.add(logLine);

    // print the scoreboard to the console (SRS 2.8c)
    System.out.println("\n----- ROUND SUMMARY -----");
    System.out.println("Round #: " + roundNumber);
    if (result.equals("DRAW")) {
        System.out.println("Result: DRAW");
    } else {
        System.out.println("Result: " + winnerLabel + " wins!");
    }
    System.out.println("\nSCOREBOARD:");
    System.out.println(" " + userName + ": " + scoreUser);
    System.out.println(" " + botName + ": " + scoreBot);
    System.out.println("-----\n");
}

// -----
// writeLogFile — called once when the player chooses to quit.
// Writes all stored log entries to a .txt file using FileWriter.
// SRS ref: 2.8a, 2.8d
// -----

void writeLogFile() {

```

```

    String timestamp =
DateTimeFormatter.ofPattern("yyyyMMdd_HH:mm:ss").format(LocalDateTime.now());

    String filename = "noughts_log_" + timestamp + ".txt";

    try {

        FileWriter fw = new FileWriter(filename, true); // true = append mode

        for (String entry : logEntries) {

            fw.write(entry + "\n");

        }

        fw.close();

        System.out.println(YELLOW + "[SUCCESS] Log saved to: " + filename + RESET);

    } catch (IOException e) {

        System.out.println(RED + "*** [ERROR] Could not save log: " + e.getMessage() + " ***" +
RESET);

    }

}

// =====

// METHOD 7 — playAgainOrQuit

// Lights Button Y (yellow) and Button X (blue), waits for one to

// be pressed. Y resets the board; X quits.

// Returns true to keep playing, false to quit.

// SRS ref: 2.9

// =====

boolean playAgainOrQuit() {

    System.out.println("-----");

    System.out.println(" Press Button Y (YELLOW LED) to play again");

    System.out.println(" Press Button X (BLUE LED)  to quit");

    System.out.println(" Or type Y / X and press Enter:");

    System.out.println("-----");

    char choice = '?';

    if (hardwareOn) {

```

```

try {

    // light both buttons with their SRS colours (SRS 2.2)

    api.setButtonLight(Button.Y, true); // yellow
    api.setButtonLight(Button.X, true); // blue

    // boolean array trick so the lambda can write to it
    boolean[] pressed = { false };
    char[] result = { '?' };

    api.enableButton(Button.Y, () -> {
        if (!pressed[0]) {
            result[0] = 'Y';
            pressed[0] = true;
        }
    });

    api.enableButton(Button.X, () -> {
        if (!pressed[0]) {
            result[0] = 'X';
            pressed[0] = true;
        }
    });

    // wait until one button is pressed
    while (!pressed[0]) {
        Thread.sleep(100);
    }

    // clean up lights and button listeners
    api.disableAllButtons();
    api.setButtonLight(Button.Y, false);
    api.setButtonLight(Button.X, false);
}

```

```

        choice = result[0];

        System.out.println("[ACTION] Button " + choice + " pressed.");

    } catch (Exception e) {

        // hardware error: fall back to keyboard

        System.out.print("[INPUT] Enter Y or X:\n> ");

        String s = in.nextLine().trim().toUpperCase();

        choice = s.isEmpty() ? '?' : s.charAt(0);

    }

} else {

    // keyboard-only fallback

    System.out.print("[INPUT] Enter Y or X:\n> ");

    String s = in.nextLine().trim().toUpperCase();

    choice = s.isEmpty() ? '?' : s.charAt(0);

}

return choice == 'Y';
}

// =====
// run — the main game loop
// Calls all 7 methods in order and keeps looping until the
// player chooses to quit.
// =====

void run() {

    // step 1: welcome screen and registration

    initAndRegister();

```

```

// step 2: choose difficulty

System.out.println();

System.out.print("[INPUT] Choose difficulty (easy / medium / hard):\n> ");

String d = in.nextLine().trim().toLowerCase();

if (d.equals("easy")) {

    difficulty = "easy";

    undoQuota = 2;

    timerSeconds = 0;

} else if (d.equals("hard")) {

    difficulty = "hard";

    undoQuota = 0;

    timerSeconds = 5;

} else {

    difficulty = "medium";

    undoQuota = 1;

    timerSeconds = 10;

}

System.out.println("[INFO] Difficulty: " + difficulty

    + " | Undos per round: " + undoQuota

    + " | Timer: " + timerSeconds + "s");


// step 3: roll dice to decide who goes first

rollDice();


// step 4: main game loop - keeps running until player chooses to quit

boolean keepPlaying = true;

while (keepPlaying) {

    // reset the board and undo counter at the start of every round

    for (int r = 0; r < 3; r++) {

        for (int c = 0; c < 3; c++) {

```

```

        board[r][c] = " ";
    }
}

undoRemaining = undoQuota;
lastUserRow = -1;
lastUserCol = -1;

// step 5: play one round until there is a winner or a draw
boolean roundOver = false;
while (!roundOver) {

    if (currentTurn.equals("user")) {
        userTurn();
        currentTurn = "bot";
    } else {
        swiftBotTurn();
        currentTurn = "user";
    }

    // after every move, check if the round has ended
    String result = checkWinOrDraw();
    if (result != null) {
        handleOutcome(result);
        saveLogAndShowScore(result);
        roundOver = true;
    }
}

// step 6: ask the player to play again or quit
keepPlaying = playAgainOrQuit();
}

// step 7: write log file to disk and say goodbye

```

```

    writeLogFile();

    System.out.println("[PROCESS] Program terminating. See you later!");
}

// =====

// SMALL HELPER METHODS

// These keep the 7 main methods readable by handling
// repeated low-level tasks in one place.

// =====

// prints the current board state to the console
void printBoard() {
    System.out.println();
    System.out.println("   Columns ->  1  2  3");
    System.out.println("Rows");
    for (int r = 0; r < 3; r++) {
        System.out.print(" " + (r + 1) + "   ");
        for (int c = 0; c < 3; c++) {
            System.out.print("[ " + board[r][c] + " ] ");
        }
        System.out.println();
    }
    System.out.println();
}

// blinks all underlights a set number of times in the given colour.
// color must be "GREEN", "RED", "BLUE", or "YELLOW"
void blinkUnderlights(String color, int times) {
    if (!hardwareOn) {
        System.out.println("[MOTION] Blink " + color + " x" + times);
        return;
    }
}

```

```

    }

    int[] rgb;

    if (color.equals("GREEN")) rgb = new int[]{ 0, 255, 0 };
    else if (color.equals("RED")) rgb = new int[]{ 255, 0, 0 };
    else if (color.equals("BLUE")) rgb = new int[]{ 0, 0, 255 };
    else rgb = new int[]{ 255, 255, 0 }; // YELLOW

    try {
        for (int i = 0; i < times; i++) {
            api.fillUnderlights(rgb);

            Thread.sleep(300);

            api.disableUnderlights();

            Thread.sleep(300);
        }
    } catch (Exception e) {
        System.out.println(RED + "*** [ERROR] Blink failed: " + e.getMessage() + " ***" + RESET);
    }
}

// spins the SwiftBot 360 degrees in place
void spinOnce() {
    if (!hardwareOn) {
        System.out.println("[MOTION] Spin once");
        return;
    }

    try {
        api.move(100, -100, 600); // spin 180 degrees

        Thread.sleep(100);

        api.move(100, -100, 600); // another 180 to complete the circle
    } catch (Exception e) {
        System.out.println(RED + "*** [ERROR] Spin failed: " + e.getMessage() + " ***" + RESET);
    }
}

```